

Tugas 4: Implementasi Shamir Secret Sharing pada Aplikasi Password Manager Terdistribusi

II4021 - Kriptografi



Disusun oleh Kelompok 9:

Alma Felicia Vielrizki - 18223112

Aulia Azka Azzahra - 18223131

Sonya Putri Fadilah - 18223138

**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
FAKULTAS SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

DAFTAR ISI

DAFTAR ISI	2
DAFTAR TABEL	4
DAFTAR GAMBAR	5
I. PENDAHULUAN	8
1.1 Latar Belakang	8
1.2 Tujuan	9
1.3 Ruang Lingkup	9
II. DASAR TEORI	10
2.1 Shamir Secret Sharing	10
2.2 AES-GCM	11
2.3 Key Derivation Function (KDF)	12
2.4 CSPRNG	12
2.5 Kriptografi Visual	13
III. PERANCANGAN DAN IMPLEMENTASI	14
3.1 Arsitektur Sistem	14
3.1.1 Penyimpanan Data pada Komponen Sistem	15
3.1.2 Tech Stack	16
3.2 Implementasi Shamir Secret Sharing	17
3.2.1 Pemilihan Bilangan Prima dan Representasi Data	17
3.2.2 Mekanisme Pembagian dan Rekonstruksi Rahasia	18
3.3 Implementasi AES-GCM	18
3.4 Implementasi KDF (Argon2id)	20
3.4.1 Mekanisme derive_key() dan Parameter Argon2id	20
3.4.2 Penyimpanan Salt dan Parameter di Sisi Klien	21
3.5 Implementasi CSPRNG	21
3.5.1 Mekanisme Pembangkitan Password (generate_password)	22
3.5.2 Pemetaan Bilangan Acak Tanpa Bias	22
3.6 Alur Pembuatan Vault	23
3.7 Alur Akses Normal	24
3.8 Alur Akses Backup	24
3.9 Implementasi Server	25
3.10 Penambahan Data Password	27
3.11 Implementasi CLI	30
3.12 Implementasi Kriptografi Visual	34
IV. PENGUJIAN DAN ANALISIS	35
4.1 Uji Pembuatan Vault	35
4.2 Uji Penyimpanan dan Perlindungan Local Share	41
4.3 Uji Akses Normal	46
4.4 Uji Penambahan Data Password	51
4.5 Uji Pengubahan dan Penghapusan Data Password	58

4.6 Uji Penyimpanan Vault di Server	62
4.7 Uji Mode Backup	67
4.8 Uji Kegagalan Pemulihan	71
4.9 Uji Kriptografi Visual	75
V. KESIMPULAN	80
DAFTAR PUSTAKA	81
LAMPIRAN	82
A. Pranala repository	82
B. Pranala video demo	82
C. Pembagian tugas	82

DAFTAR TABEL

Tabel 1. Penyimpanan Data pada Komponen Sistem	15
Tabel 2. Tech Stack yang Digunakan	16
Tabel 3. Perbedaan Mode Normal dan Mode Backup	25
Tabel 4. Endpoint API Server	26
Tabel 5. Struktur Tabel users pada SQLite	26
Tabel 6. Struktur Minimal Data Password	27
Tabel 7. Penjelasan Tampilan Program	30

DAFTAR GAMBAR

Gambar 3.1 Arsitektur Sistem Password Manager Terdistribusi	15
Gambar 3.2 Flowchart Implementasi Kriptografi Visual	34
Gambar 4.1 Output pembuatan vault baru	37
Gambar 4.2 Output pembagian master key menjadi tiga share	38
Gambar 4.3 Output Unit Test Pembuatan Vault dan Pembagian Share SSS (2,3)	38
Gambar 4.4 Tampilan recovery share	39
Gambar 4.5 Isi basis data server setelah pembuatan vault	40
Gambar 4.6 Berkas demo_user.json menampilkan Local Share Terenkripsi di Sisi Klien	42
Gambar 4.7 Konfigurasi KDF Argon2id sebagai Pelindung Local Share	42
Gambar 4.8 Berhasil Membuka Vault Menggunakan Master Password yang Benar	44
Gambar 4.9 Akses Ditolak Akibat Input Master Password yang Salah	45
Gambar 4.10 Berhasil masuk dengan master password	47
Gambar 4.11 Hasil pengujian masuk rekonstruksi master key menggunakan script	48
Gambar 4.12 Hasil pengujian masuk dalam CLI	49
Gambar 4.13 Tampilan vault masih kosong	50
Gambar 4.14 Tampilan vault yang diisi	50
Gambar 4. 15 User gagal masuk karena master password salah	51
Gambar 4.16 Input password baru	53
Gambar 4.17 Password baru telah masuk ke daftar password	53
Gambar 4.18 Input password baru menggunakan CSPRNG	54
Gambar 4.19 Input password baru menggunakan CSPRNG	55
Gambar 4.20 Hasil nonce sebelum ditambahkan password	56
Gambar 4.21 Hasil nonce setelah ditambahkan password	56
Gambar 4.22 Hasil file JSON lokal sebelum ditambahkan password	57
Gambar 4.23 Hasil file JSON lokal setelah ditambahkan password	57
Gambar 4.24 Mengubah password	59
Gambar 4.25 Tampilan password yang telah diubah	59
Gambar 4.26 Menghapus password	60
Gambar 4.27 Tampilan list password	61
Gambar 4.28 Mode backup tidak dapat mengubah/ menghapus password	62
Gambar 4.29 Output pengecekan tipe data vault_blob pada database server	65
Gambar 4.30 Output pengecekan plaintext data vault pada server	65
Gambar 4.31 Output struktur tabel users dan pengecekan kolom terlarang	66
Gambar 4.32 Tampilan status server offline	68
Gambar 4.33 Output vault berhasil dibuka dengan local share dan recovery share	68
Gambar 4.34 Output pembukaan vault pada mode backup saat server offline	69
Gambar 4.35 Tampilan daftar password pada mode backup	70
Gambar 4.36 Tampilan mode BACKUP (READ-ONLY) dan menu tanpa operasi tulis	71
Gambar 4.37 Kegagalan Rekonstruksi Kunci Akibat Nilai Recovery Share Tidak Valid	73
Gambar 4.38 Kegagalan Dekripsi Backup Vault Akibat Deteksi Modifikasi Data	74

Gambar 4.39 Antarmuka CLI Tetap Bersih dari Informasi Sandi Setelah Terjadi Kegagalan Proses Dekripsi	75
Gambar 4.40 QR code awal dari recovery share	76
Gambar 4.41 Visual share 1 dan visual share 2 hasil pembagian QR code	77
Gambar 4.42 QR code hasil penggabungan visual share	78
Gambar 4.43 Hasil pemindaian QR code rekonstruksi	79

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Alma Felicia Vielrizki
18223112



Aulia Azka Azzahra
18223131



Sonya Putri Fadilah
18223138

I. PENDAHULUAN

1.1 Latar Belakang

Pengelolaan kata sandi (*password*) secara aman merupakan salah satu tantangan penting dalam keamanan siber modern. Pengguna umumnya memiliki banyak akun digital, seperti email, media sosial, layanan penyimpanan data, hingga portal akademik. Setiap akun sebaiknya menggunakan *password* yang berbeda untuk mengurangi risiko kebocoran berantai. Namun, semakin banyak *password* yang digunakan, semakin sulit pula pengguna mengingat dan mengelolanya secara manual.

Password manager dapat digunakan untuk menyimpan berbagai data akun secara terpusat dalam sebuah vault. Akan tetapi, sistem password manager yang bergantung sepenuhnya pada satu server memiliki risiko *single point of failure*. Jika server mengalami kompromi, kehilangan data, atau tidak dapat diakses, maka keamanan dan ketersediaan vault pengguna dapat terganggu. Oleh karena itu, dibutuhkan rancangan *password* manager yang tidak hanya mampu menyimpan *password*, tetapi juga menjaga agar kunci pembuka vault tidak berada pada satu pihak saja.

Pada tugas ini, dibangun aplikasi *password manager* terdistribusi dengan pendekatan kriptografi. Sistem menggunakan arsitektur *client-server*, di mana proses kriptografi utama dilakukan di sisi klien, sedangkan server hanya menyimpan data terenkripsi dan data pendukung. *Master key* untuk membuka vault dibagi menggunakan *Shamir Secret Sharing* dengan skema ambang batas (2,3), sehingga minimal dua share valid diperlukan untuk membuka vault. Dengan rancangan ini, server tidak menyimpan *master key* secara utuh maupun isi vault dalam bentuk asli, sehingga sistem lebih sesuai dengan prinsip *zero-knowledge server* dan tetap mendukung pemulihan akses melalui mode *backup*.

1.2 Tujuan

Tujuan dari pengerjaan tugas ini adalah sebagai berikut.

1. Mengimplementasikan aplikasi *password manager* terdistribusi berbasis *client-server* untuk menyimpan data akun pengguna dalam bentuk vault terenkripsi.
2. Mengimplementasikan skema *Shamir Secret Sharing* dengan ambang batas (2,3) untuk membagi *master key* menjadi tiga bagian, yaitu *local share*, *server share*, dan *recovery share*.
3. Mengimplementasikan enkripsi vault menggunakan AES-128-GCM dan enkripsi *local share* menggunakan AES-256-GCM.
4. Mengimplementasikan *Key Derivation Function* (KDF) Argon2id untuk menurunkan kunci dari *master password*.
5. Menyediakan dua mode akses vault, yaitu mode normal menggunakan *local share* dan *server share*, serta mode *backup* menggunakan *local share* dan *recovery share*.
6. Mengimplementasikan pembangkitan *password* otomatis menggunakan *Cryptographically Secure Pseudorandom Number Generator* (CSPRNG).
7. Mengimplementasikan server berbasis SQLite yang hanya menyimpan data terenkripsi dan data pendukung, tanpa menyimpan *master key*, *local share*, *recovery share*, maupun isi vault dalam bentuk asli.
8. Mengimplementasikan fitur bonus kriptografi visual untuk merepresentasikan *recovery share* dalam bentuk *QR code* dan membaginya menjadi dua *visual share*.

1.3 Ruang Lingkup

Ruang lingkup implementasi pada tugas ini adalah sebagai berikut.

1. Aplikasi yang dibangun merupakan *password manager* terdistribusi berbasis *client-server* yang dijalankan secara lokal.

2. Antarmuka pengguna dibuat dalam bentuk *Command Line Interface* (CLI), sehingga seluruh proses pembuatan vault, akses vault, penambahan data *password*, mode *backup*, dan fitur bantuan dilakukan melalui terminal.
3. Data akun yang disimpan di dalam vault mencakup nama layanan, *username* atau email, *password*, dan catatan opsional.
4. Proses kriptografi utama dilakukan di sisi klien, meliputi pembangkitan *master key*, pembagian *master key* dengan *Shamir Secret Sharing*, enkripsi/dekripsi vault, enkripsi/dekripsi *local share*, serta pembangkitan password otomatis.
5. Server digunakan sebagai media penyimpanan berbasis SQLite untuk menyimpan *server share*, vault terenkripsi, *nonce vault*, dan metadata pengguna. Server tidak menyimpan *master key*, *local share*, *recovery share*, maupun isi vault dalam bentuk *plaintext*.
6. Sistem mendukung dua mode akses, yaitu mode normal menggunakan kombinasi *local share* dan *server share*, serta mode *backup* menggunakan kombinasi *local share* dan *recovery share*. Pada mode backup, pengguna hanya dapat melihat data *password* dan tidak dapat menambah, mengubah, atau menghapus data.
7. Implementasi fitur bonus mencakup kriptografi visual untuk *recovery share*, yaitu mengubah *recovery share* menjadi QR code dan membaginya menjadi dua *visual share*.
8. Sistem ini tidak mencakup fitur sinkronisasi *multi-device* secara nyata, ekstensi browser, aplikasi *mobile*, manajemen akun pengguna tingkat produksi, maupun *deployment* server publik.

II. DASAR TEORI

2.1 Shamir Secret Sharing

Shamir Secret Sharing (SSS) adalah skema kriptografi yang diperkenalkan oleh Adi Shamir pada 1979 ^[1], yang memungkinkan sebuah secret S dibagi menjadi n share sedemikian sehingga minimal t share dapat merekonstruksi S , sementara kurang dari t share tidak memberikan informasi apapun. Skema ini disebut (t, n) -threshold scheme.

Pada tugas ini digunakan skema (2, 3): $n = 3$ share dibangkitkan dan $t = 2$ share sudah cukup untuk rekonstruksi.

SSS dibangun di atas sifat bahwa polinomial berderajat $t-1$ ditentukan secara unik oleh t titik berbeda. Untuk skema (2, 3), digunakan polinomial berderajat 1 (linear) atas lapangan hingga modulo bilangan prima p :

$$f(x) = a_0 + a_1x \quad (\text{mod } p)$$

di mana $a_0 = S$ (*secret/master key*), a_1 adalah koefisien acak yang dipilih seragam dari $\{1, \dots, p-1\}$, dan p adalah bilangan prima yang lebih besar dari S dan n . Tiga *share* dihasilkan sebagai $(1, f(1))$, $(2, f(2))$, $(3, f(3))$. Contoh: jika $S = 1234$, $a_1 = 7$, dan $p = 1301$, maka $f(x) = 1234 + 7x \pmod{1301}$ menghasilkan *share* $(1, 1241)$, $(2, 1248)$, $(3, 1255)$.

Rekonstruksi secret dari dua share (x_1, y_1) dan (x_2, y_2) dilakukan menggunakan interpolasi Lagrange ^[1]:

$$S = f(0) = y_1 \cdot (0-x_2) / (x_1-x_2) + y_2 \cdot (0-x_1) / (x_2-x_1) \quad (\text{mod } p)$$

Pembagian dihitung sebagai perkalian invers modular menggunakan Teorema Kecil Fermat: $b^{-1} \equiv b^{p-2} \pmod{p}$. Verifikasi dengan contoh di atas: $f(0) = 1241 \cdot 2 - 1248 \pmod{1301} = 1234 = S$. Sifat *perfect secrecy* terjamin secara informasi-teoritis karena satu share tunggal tidak membocorkan informasi apapun tentang S ^[1].

2.2 AES-GCM

Advanced Encryption Standard (AES) adalah algoritma enkripsi simetris yang ditetapkan NIST pada 2001 ^[2]. Dalam mode Galois/Counter Mode (GCM) ^[3], AES menghasilkan *Authenticated Encryption with Associated Data* (AEAD) yang menjamin kerahasiaan, integritas, dan keaslian data sekaligus. GCM bekerja dalam dua lapisan: (1) *Counter Mode* yang mengenkripsi *plaintext* dengan keystream hasil AES(key, nonce || counter), dan (2) *GHASH* yang menghasilkan authentication tag 128-bit melalui perkalian di $\text{GF}(2^{128})$, mencakup *ciphertext* dan data tambahan (AAD).

Pada sistem ini digunakan dua varian: AES-128-GCM untuk enkripsi vault (kunci 128-bit sesuai ukuran *master key*) dan AES-256-GCM untuk enkripsi *local share* (kunci 256-bit turunan dari master password via KDF, memberikan margin keamanan ekstra). Setiap enkripsi menggunakan nonce baru 96-bit yang dibangkitkan via `secrets.token_bytes(12)`, karena *nonce reuse* dalam AES-GCM memungkinkan pemulihan XOR plaintext dan pemalsuan authentication tag sebuah kelemahan kritis ^[3].

2.3 Key Derivation Function (KDF)

Argon2id adalah fungsi penurunan kunci dari *password* yang memenangkan *Password Hashing Competition* (PHC) 2015 ^[4]. Argon2id merupakan varian hibrida: menggabungkan Argon2i (akses memori *data-independent*, tahan *side-channel attack*) dan Argon2d (akses bergantung data, tahan serangan GPU/ASIC). Tiga parameter utama yang dikonfigurasi adalah *time_cost* (jumlah iterasi), *memory_cost* (ukuran memori dalam KB), dan *parallelism*. Implementasi ini menggunakan *time_cost* = 3, *memory_cost* = 65536 KB (64 MB), *parallelism* = 4, dan salt acak 16 byte, sesuai rekomendasi OWASP untuk autentikasi interaktif.

Dibandingkan alternatif lain: PBKDF2 hanya menambah beban CPU tanpa memori sehingga rentan terhadap GPU; bcrypt menggunakan memori tetap ~4 KB yang tidak dapat diskalakan; scrypt memperkenalkan biaya memori namun rentan terhadap *time-memory trade-off attack*. Argon2id unggul karena parameter memori tinggi yang fleksibel dan ketahanan terhadap berbagai jenis serangan secara bersamaan ^[4].

2.4 CSPRNG

Cryptographically Secure Pseudorandom Number Generator (CSPRNG) adalah generator bilangan acak yang memenuhi dua syarat kriptografis: (1) output tidak dapat dibedakan dari distribusi seragam sejati, dan (2) output tidak dapat diprediksi meskipun sejumlah output sebelumnya diketahui ^[5]. Python menyediakan modul *secrets* yang menggunakan sumber entropi sistem operasi (`/dev/urandom` pada Linux atau `CryptGenRandom` pada Windows) sebagai CSPRNG, sehingga aman untuk keperluan kriptografi.

Pada sistem ini CSPRNG digunakan untuk dua keperluan: (1) pembangkitan *master key* 128-bit dan nonce 96-bit menggunakan `secrets.token_bytes(n)`, dan (2) pembangkitan *password* otomatis. Untuk pembangkitan *password*, dipilih karakter secara acak dari charset yang dikonfigurasi (huruf besar, huruf kecil, angka, dan/atau simbol). Pemilihan dilakukan menggunakan `secrets.randbelow(charset)` agar setiap karakter terpilih dengan probabilitas seragam tanpa bias modular berbeda dengan pendekatan `random.randint() % len(charset)` yang menghasilkan distribusi tidak merata jika panjang charset bukan pembagi dari 2^k . Panjang dan komposisi karakter *password* dapat dikonfigurasi sesuai kebutuhan pengguna.

2.5 Kriptografi Visual

Visual Secret Sharing (VSS) adalah bentuk khusus *secret sharing* yang diperkenalkan oleh Naor dan Shamir pada 1994 ^[6], di mana secret berupa gambar dibagi menjadi n share berupa gambar acak (*noise*). Rekonstruksi dilakukan secara visual dengan menumpangkan (*overlay*) share tanpa komputasi apapun. Pada skema (2, 2), setiap pixel secret dipecah menjadi dua subpixel pada masing-masing share menggunakan tabel distribusi: pixel putih dibagi menjadi dua subpixel identik (keduanya hitam atau keduanya putih), sedangkan pixel hitam dibagi menjadi dua subpixel berlawanan (satu hitam satu putih). Saat kedua *share* ditumpangkan, pixel hitam selalu menghasilkan area gelap, sementara pixel putih menghasilkan area abu-abu perbedaan kontras inilah yang memunculkan gambar *secret*.

QR code dipilih sebagai media *recovery share* karena sifatnya yang binary (modul hitam-putih) sesuai dengan model pixel VSS, serta memiliki kemampuan koreksi error (*error correction*) bawaan hingga tingkat H (30% kerusakan data dapat dipulihkan). Hal ini penting karena proses *overlay* VSS menambah *noise visual* pada gambar akhir. Alur implementasi: *recovery share* diubah menjadi QR code, lalu QR code tersebut diproses VSS (2, 2) menghasilkan dua gambar *share* yang masing-masing tampak sebagai *noise* acak. Rekonstruksi dilakukan dengan menumpangkan kedua gambar *share* secara pixel-per-pixel menggunakan operasi OR, sehingga QR *code* muncul kembali dan dapat dipindai untuk memperoleh *recovery share*.

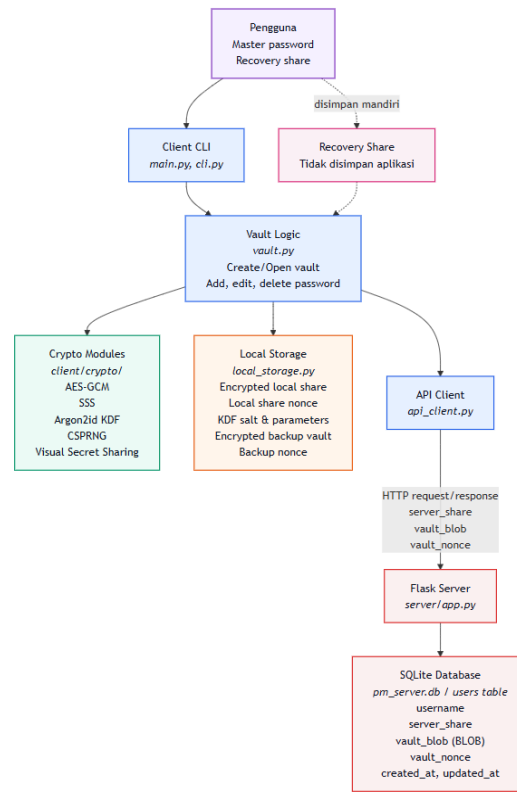
III. PERANCANGAN DAN IMPLEMENTASI

3.1 Arsitektur Sistem

Sistem *password manager* ini menggunakan arsitektur *client-server*. Klien berperan sebagai pusat operasi kriptografi, sedangkan server berperan sebagai media penyimpanan data terenkripsi dan data pendukung yang diperlukan pada mode akses normal. Dengan rancangan ini, data sensitif seperti *master password*, *master key*, *local share*, *recovery share*, dan isi vault dalam bentuk *plaintext* tidak pernah disimpan di server.

Pada sisi klien, aplikasi dijalankan melalui *Command Line Interface* (CLI). Klien menangani proses pembuatan vault, rekonstruksi *master key*, enkripsi dan dekripsi vault, perlindungan *local share*, pembaruan backup vault lokal, serta komunikasi dengan server. Sementara itu, server menggunakan Flask dan SQLite untuk menyimpan *server share*, vault terenkripsi, *nonce vault*, dan metadata pengguna [7]. Server tidak melakukan proses enkripsi, dekripsi, maupun rekonstruksi *master key*.

Alur umum arsitektur sistem ditunjukkan pada berikut.



Gambar 3.1 Arsitektur Sistem *Password Manager* Terdistribusi

3.1.1 Penyimpanan Data pada Komponen Sistem

Berikut adalah data yang disimpan pada masing-masing komponen sistem.

Tabel 1. Penyimpanan Data pada Komponen Sistem

Lokasi Penyimpanan	Data yang Disimpan	Keterangan
Klien	<i>Local share</i> terenkripsi	<i>Local share</i> tidak disimpan dalam bentuk asli. Data ini dienkripsi menggunakan AES-256-GCM dengan kunci turunan dari <i>master password</i> .
Klien	Nonce enkripsi <i>local share</i>	Digunakan untuk mendekripsi <i>local share</i> ketika pengguna memasukkan <i>master password</i> .

Klien	Salt KDF dan parameter KDF	Digunakan agar sistem dapat menurunkan kembali kunci yang sama dari <i>master password</i> saat pengguna membuka vault.
Klien	Backup vault lokal terenkripsi	Digunakan pada mode <i>backup</i> ketika server tidak dapat diakses.
Klien	Nonce backup vault	Digunakan untuk mendekripsi backup vault lokal.
Server	<i>Server share</i>	Salah satu share hasil pembagian <i>master key</i> menggunakan <i>Shamir Secret Sharing</i> . Satu share saja tidak cukup untuk membuka vault.
Server	Vault terenkripsi	Isi vault yang telah dienkripsi menggunakan AES-128-GCM di sisi klien dan disimpan sebagai BLOB pada SQLite.
Server	Nonce vault	Nonce yang digunakan pada enkripsi vault terakhir.
Server	Metadata pengguna	Data pendukung seperti <i>username</i> , waktu pembuatan, dan waktu pembaruan vault.
Pengguna	<i>Recovery share</i>	<i>Share</i> cadangan yang ditampilkan kepada pengguna dan harus disimpan secara mandiri. Aplikasi tidak menyimpan <i>recovery share</i> .

3.1.2 Tech Stack

Berikut adalah *tech stack* yang digunakan dalam implementasi sistem.

Tabel 2. *Tech Stack* yang Digunakan

Komponen	Keterangan
Bahasa pemrograman	Python 3.11+
Antarmuka pengguna	<i>Command Line Interface</i> (CLI) berbasis <i>standard library</i> Python
Server	Flask REST API

Database server	SQLite
Komunikasi <i>client-server</i>	HTTP <i>request</i> menggunakan <i>library requests</i>
<i>Shamir Secret Sharing</i>	Implementasi sendiri menggunakan interpolasi Lagrange pada <i>finite field</i>
Enkripsi vault	AES-128-GCM menggunakan <i>library cryptography</i>
Enkripsi <i>local share</i>	AES-256-GCM menggunakan <i>library cryptography</i>
KDF	Argon2id menggunakan <i>library argon2-cffi</i>
CSPRNG	Modul secrets dan sumber acak sistem operasi
QR Code	<i>Library qrcode</i>
Kriptografi visual	<i>Library Pillow</i> dan implementasi sendiri

3.2 Implementasi Shamir Secret Sharing

File: `client/crypto/sss.py`

Implementasi Shamir's Secret Sharing (SSS) pada modul ini bertujuan untuk membagi *master key* menjadi beberapa bagian (*shares*) dengan skema ambang batas $(t, n) = (2, 3)$. Dengan skema ini, *master key* dipecah menjadi tiga bagian *local share*, *server share*, dan *recovery share* di mana minimal dua *share* diperlukan untuk merekonstruksi kembali kunci asli. Seluruh operasi aritmetika dilakukan di dalam *finite field* $GF(p)$ untuk menjamin keamanan informasi secara teoretis.

3.2.1 Pemilihan Bilangan Prima dan Representasi Data

Landasan utama dari implementasi ini adalah penggunaan bilangan prima p sebagai modulus lapangan. Kode ini menggunakan bilangan prima 256-bit:

$$p = 2^{256} - 2^{32} - 977$$

Pemilihan prima ini memastikan bahwa nilai rahasia (128-bit untuk AES-128) selalu lebih kecil dari p , sehingga seluruh elemen rahasia dapat direpresentasikan secara unik dalam lapangan modular. Sebelum diproses oleh algoritma SSS, *master key* yang bertipe *bytes* dikonversi menjadi integer besar melalui fungsi `int.from_bytes(secret, byteorder="big")`. Setelah proses rekonstruksi selesai, integer tersebut dikonversi kembali menjadi *bytes* 32-byte dan dipotong (*slicing*) untuk mengambil 16-byte terakhir guna mendapatkan kembali kunci AES-128 yang presisi.

3.2.2 Mekanisme Pembagian dan Rekonstruksi Rahasia

Proses pembagian rahasia dilakukan melalui fungsi `generate_shares()` dengan membentuk polinomial acak berderajat satu ($t-1$), yaitu $f(x) = s + a_1x \pmod{p}$. Di sini, s adalah *master key* dan a_1 adalah koefisien acak yang dibangkitkan menggunakan `secrets.randbelow()`. Fungsi ini kemudian menghasilkan tiga titik koordinat ($i, f(i)$) untuk $i = 1, 2, 3$ yang dikonversi ke dalam format heksadesimal agar mudah disimpan atau ditransmisikan.

Untuk memulihkan rahasia, fungsi `reconstruct_secret()` menerapkan **Interpolasi Lagrange** pada titik $x = 0$. Karena operasi dilakukan dalam lapangan modular, pembagian polinomial diselesaikan dengan perkalian invers modular menggunakan algoritma *Extended Euclidean* melalui fungsi `_mod_inverse()`. Jika setidaknya dua *share* yang diberikan valid, perhitungan $f(0)$ akan menghasilkan kembali *master key* yang asli. Namun, jika *share* yang digunakan salah, hasil rekonstruksi akan meleset dan menyebabkan kegagalan autentikasi pada proses dekripsi vault.

3.3 Implementasi AES-GCM

File: `client/crypto/aes_gcm.py`

Modul `aes_gcm.py` mengimplementasikan algoritma *Advanced Encryption Standard* dengan mode *Galois/Counter Mode* (AES-GCM) untuk menjamin kerahasiaan (*confidentiality*) sekaligus integritas data (*authenticity*).

Penggunaan mode GCM sangat krusial karena sistem dapat mendeteksi jika *ciphertext* atau *nonce* telah dimodifikasi secara sengaja, yang akan menyebabkan proses dekripsi gagal secara otomatis. Dalam pengembangannya, AES-GCM digunakan untuk dua fungsi utama dengan tingkat keamanan kunci yang berbeda:

- **Enkripsi Vault (`encrypt_vault`):** Fungsi ini digunakan untuk mengamankan seluruh isi data akun dalam vault menggunakan standar **AES-128-GCM**. Kunci yang digunakan adalah *master key* 16-byte hasil rekonstruksi SSS. Hasil enkripsi berupa gabungan antara *ciphertext* dan *authentication tag* sepanjang 16-byte yang disematkan di akhir (`ciphertext || tag`).
- **Enkripsi Local Share (`encrypt_local_share`):** Untuk memberikan perlindungan ekstra pada *local share* yang disimpan di perangkat, digunakan standar **AES-256-GCM**. Kunci yang digunakan adalah kunci turunan 32-byte yang dihasilkan oleh Argon2id dari *master password* pengguna. Hal ini memastikan bahwa meskipun seseorang memiliki akses fisik ke file *local share*, mereka tidak dapat membacanya tanpa kunci turunan yang tepat.

Keamanan skema AES-GCM dalam modul ini sangat bergantung pada penanganan *nonce* (*number used once*). Sesuai dengan spesifikasi, sistem selalu membangkitkan *nonce* baru yang unik berukuran 12-byte (96-bit) setiap kali proses enkripsi dilakukan menggunakan `os.urandom(_NONCE_SIZE)`. Penggunaan *nonce* yang acak dan baru untuk setiap enkripsi sangat penting guna menghindari serangan kriptanalisis yang muncul jika kombinasi kunci dan *nonce* yang sama digunakan berulang kali. *Nonce* ini nantinya disimpan oleh server (untuk vault) atau klien (untuk *local share*) agar dapat digunakan kembali sebagai parameter wajib saat proses dekripsi melalui fungsi `decrypt_vault` atau `decrypt_local_share`.

3.4 Implementasi KDF (Argon2id)

File: `client/crypto/kdf.py`

Untuk mengubah *master password* pengguna menjadi kunci kriptografi yang kuat, aplikasi ini mengimplementasikan *Key Derivation Function* (KDF) menggunakan algoritma Argon2id. Pemilihan Argon2id didasarkan pada ketahanannya yang tinggi terhadap serangan *brute-force* melalui pemanfaatan biaya waktu (*time cost*) dan biaya memori (*memory cost*), serta kemampuannya dalam menangkal serangan berbasis GPU maupun *side-channel*. Modul ini bertanggung jawab menghasilkan kunci 32-byte (256-bit) yang nantinya digunakan khusus untuk proses enkripsi dan dekripsi *local share* melalui AES-256-GCM.

3.4.1 Mekanisme `derive_key()` dan Parameter Argon2id

Fungsi utama dalam modul ini, `derive_key()`, bekerja dengan menerima *master password* dalam bentuk *plaintext* dan memprosesnya menggunakan pustaka `argon2-cffi`. Untuk menjaga keseimbangan antara keamanan yang optimal dan performa aplikasi, dipilih parameter default yang merujuk pada rekomendasi OWASP 2023:

- **Time Cost:** 3 iterasi, untuk menentukan durasi komputasi guna mempersulit serangan *brute-force*.
- **Memory Cost:** 65536 KiB (64 MB), untuk memastikan penggunaan memori yang signifikan sehingga sulit dijalankan secara massal pada perangkat keras khusus.
- **Parallelism:** 4 *thread*, untuk memanfaatkan kemampuan pemrosesan paralel pada CPU.
- **Hash Length:** 32 byte, untuk menghasilkan kunci yang sesuai dengan kebutuhan standar AES-256.

Proses penurunan kunci ini bersifat dinamis; jika fungsi dipanggil tanpa menyertakan *salt*, maka sistem akan secara otomatis membangkitkan *salt* baru sepanjang 16-byte menggunakan `secrets.token_bytes(_SALT_SIZE)`.

3.4.2 Penyimpanan Salt dan Parameter di Sisi Klien

Keamanan skema ini sangat bergantung pada penggunaan *salt* yang unik untuk setiap pengguna guna menghindari serangan *rainbow table*. Oleh karena itu, cara penyimpanan data pendukung KDF diatur sebagai berikut:

- **Penyimpanan Lokal:** Setelah kunci diturunkan saat pembuatan *vault*, nilai *salt* dan parameter KDF (seperti *time_cost* dan *memory_cost*) disimpan secara permanen di dalam penyimpanan lokal klien.
- **Transparansi Parameter:** *Salt* dan parameter KDF tidak bersifat rahasia, sehingga dapat disimpan dalam bentuk teks biasa di sisi klien. Data ini diperlukan agar sistem dapat menurunkan kunci yang identik setiap kali pengguna ingin membuka *vault* kembali (login).
- **Keamanan Password:** Penting untuk dicatat bahwa *master password* pengguna dan kunci hasil penurunan (*kdf_key*) tidak pernah disimpan secara permanen di penyimpanan fisik. *Kdf_key* hanya berada sementara di memori selama sesi aplikasi berlangsung untuk keperluan dekripsi *local share*.

3.5 Implementasi CSPRNG

File: `client/crypto/csprng.py`

Untuk mendukung fitur pembangkitan password otomatis yang aman, aplikasi ini mengimplementasikan *Cryptographically Secure Pseudorandom Number Generator* (CSPRNG) melalui modul `secrets` bawaan Python. Berbeda dengan generator angka acak standar yang bersifat deterministik, CSPRNG memanfaatkan entropi dari sistem operasi (*kernel CSPRNG*) untuk menghasilkan nilai yang tidak dapat diprediksi, bahkan jika status awal generator diketahui. Hal ini memastikan bahwa *password* yang dihasilkan memiliki tingkat keamanan tinggi dan layak digunakan untuk keperluan kriptografi.

3.5.1 Mekanisme Pembangkitan Password (`generate_password`)

Fungsi `generate_password()` dirancang untuk membangkitkan string acak berdasarkan parameter panjang dan kategori karakter yang diinginkan oleh pengguna. Proses diawali dengan membangun sebuah *pool* karakter yang terdiri dari huruf besar, huruf kecil, angka, dan simbol sesuai pilihan. Untuk menjamin kompleksitas *password*, fungsi ini menerapkan aturan verifikasi ketat di mana minimal satu karakter dari setiap kategori yang dipilih wajib disertakan dalam *password* akhir.

Alur kerjanya dibagi menjadi tiga tahap utama:

- **Penjaminan Kategori:** Mengambil satu karakter acak dari setiap kategori yang diaktifkan untuk memenuhi syarat kompleksitas minimal.
- **Pengisian Sisa Slot:** Mengisi sisa panjang *password* yang diminta dengan mengambil karakter secara acak dari gabungan seluruh kategori yang dipilih.
- **Pengacakan Akhir:** Seluruh karakter yang telah terkumpul diacak ulang urutannya menggunakan metode *Fisher-Yates shuffle* melalui `secrets.SystemRandom().shuffle()`. Langkah ini memastikan bahwa karakter-karakter yang diwajibkan tadi tidak selalu berada di posisi awal, sehingga pola *password* tidak dapat ditebak.

3.5.2 Pemetaan Bilangan Acak Tanpa Bias

Salah satu aspek krusial dalam pembangkitan *password* adalah memastikan bahwa setiap karakter dalam *pool* memiliki peluang yang sama untuk terpilih. Modul ini menggunakan fungsi `secrets.randbelow(n)` untuk menentukan indeks karakter yang akan diambil. Fungsi ini menjamin pemetaan bilangan acak yang bersifat *unbiased* (tanpa bias), yang berarti tidak ada kecenderungan statistik yang memihak pada karakter atau rentang angka tertentu.

Sebagai contoh, jika sebuah *pool* memiliki 90 karakter, fungsi akan membangkitkan integer seragam dalam rentang [0, 89] dengan probabilitas terpilihnya setiap karakter adalah tepat 1/90. Dengan pemetaan indeks yang adil ini, ruang kunci (*key space*) yang dihasilkan menjadi maksimal; misalnya, untuk password dengan panjang 16 karakter dari *pool* tersebut, terdapat sekitar $90^{16} \approx 1,85 \times 10^{31}$ kemungkinan kombinasi yang berbeda, sehingga sangat mustahil untuk ditembus melalui serangan pencarian paksa (*brute-force*).

3.6 Alur Pembuatan Vault

File: `client/cli.py`, `client/vault.py`, `client/local_storage.py`, `client/api_client.py`, `client/crypto/aes_gcm.py`, `client/crypto/sss.py`, `client/crypto/kdf.py`, `server/app.py`, `server/database.py`, dan `server/schema.sql`.

Saat pengguna membuat vault baru, sistem membangkitkan *master key* 128-bit secara acak menggunakan CSPRNG, lalu membuat vault kosong dan mengenkripsinya dengan AES-128-GCM. Master key kemudian dibagi menjadi 3 *share* menggunakan *Shamir Secret Sharing* (2,3) yang artinya cukup 2 *share* untuk merekonstruksi kunci.

```
master_key = secrets.token_bytes(MASTER_KEY_SIZE)          # 128-bit
random
shares = generate_shares(master_key, SSS_THRESHOLD, SSS_TOTAL) #
(2,3)
# shares[0] = local, shares[1] = server, shares[2] = recovery
```

Local share dienkripsi lagi menggunakan kunci yang diturunkan dari *master password* via Argon2id KDF, sebelum disimpan di client. Server share dikirim ke server bersama vault terenkripsi. *Recovery share* ditampilkan sekali ke pengguna untuk disimpan sendiri secara offline. *Master key* tidak pernah disimpan melainkan hanya ada di memori saat proses berlangsung.

3.7 Alur Akses Normal

File: `client/cli.py`, `client/vault.py`, `client/local_storage.py`, `client/api_client.py`, `client/crypto/aes_gcm.py`, `client/crypto/sss.py`, `client/crypto/kdf.py`, `server/app.py`, dan `server/database.py`.

Pada akses normal (server aktif), sistem membutuhkan dua *share*: *local share* dari file lokal dan *server share* dari server.

```
kdf_key, _, _ = derive_key(master_password, kdf_salt, kdf_params)
local_share = json.loads(decrypt_local_share(enc_local_share, nonce,
kdf_key))

server_data = api.fetch_server_data(username)
master_key = reconstruct_secret([local_share,
server_data["server_share"]])
vault = json.loads(decrypt_vault(vault_blob, vault_nonce,
master_key))
```

Master password digunakan untuk mendekripsi *local share*, lalu dikombinasikan dengan server share untuk merekonstruksi *master key* via SSS. *Master key* kemudian mendekripsi vault. Mode ini mendukung operasi penuh: baca, tambah, ubah, dan hapus *entry password*.

3.8 Alur Akses Backup

File: `client/cli.py`, `client/vault.py`, `client/local_storage.py`, `client/crypto/aes_gcm.py`, `client/crypto/sss.py`, dan `client/crypto/kdf.py`.

Mode *backup* digunakan saat server tidak dapat diakses. Alih-alih *server share*, pengguna menginput *recovery share* secara manual. Sumber vault berasal dari backup lokal.

```
master_key = reconstruct_secret([local_share, recovery_share])
vault = json.loads(decrypt_vault(enc_backup_vault,
backup_nonce, master_key))
```


Mode ini bersifat *read-only* dan tidak ada operasi tulis karena data tidak bisa disinkronkan ke server. Data yang ditampilkan adalah kondisi vault pada saat terakhir kali tersimpan.

Tabel 3. Perbedaan Mode Normal dan Mode *Backup*

Aspek	Mode Normal	Mode Backup
Kombinasi <i>share</i>	<i>Local share + server share</i>	<i>Local share + recovery share</i>
Sumber vault	Vault terenkripsi dari server	<i>Backup</i> vault terenkripsi di klien
Operasi yang didukung	Melihat, menambah, mengubah, dan menghapus data <i>password</i>	Melihat data <i>password</i> saja
Pembaruan data	Vault dienkripsi ulang dengan nonce baru, lalu disimpan ke server dan <i>backup</i> lokal	Tidak melakukan pembaruan data
Tujuan utama	Akses biasa	Akses darurat saat server tidak dapat diakses

3.9 Implementasi Server

File: `server/app.py`, `server/database.py`, `server/schema.sql`, `server/config.py`, dan `client/api_client.py`.

Server pada sistem ini diimplementasikan menggunakan Flask sebagai REST API dan SQLite sebagai basis data. Server berperan sebagai media penyimpanan data yang diperlukan untuk mode normal, yaitu *server share*, vault terenkripsi, nonce vault, dan metadata pengguna. Seluruh proses kriptografi, seperti rekonstruksi master key, enkripsi vault, dan dekripsi vault tetap dilakukan di sisi klien.

Komunikasi antara klien dan server dilakukan melalui HTTP *request*. Vault terenkripsi dikirim dari klien ke server dalam format base64 agar dapat dikirim melalui JSON, kemudian disimpan di SQLite sebagai BLOB ^[8].

Ketika klien membutuhkan vault pada mode normal, server hanya mengirimkan kembali *server share*, vault terenkripsi, dan nonce vault. Server tidak memiliki *local share* atau *recovery share*, sehingga satu *server share* saja tidak cukup untuk merekonstruksi *master key*.

Tabel 4. Endpoint API Server

<i>Endpoint</i>	<i>Method</i>	Fungsi
/register	POST	Mendaftarkan pengguna baru dan menyimpan data awal vault, yaitu username, <i>server share</i> , vault terenkripsi, dan nonce vault.
/vault/<username>	GET	Mengirimkan <i>server share</i> , vault terenkripsi, dan nonce vault kepada klien untuk mode akses normal.
/vault/<username>	PUT	Memperbarui vault terenkripsi dan nonce vault setelah pengguna menambah, mengubah, atau menghapus data <i>password</i> pada mode normal.
/ping	GET	Mengecek apakah server sedang aktif atau tidak. <i>Endpoint</i> ini digunakan klien untuk menentukan ketersediaan mode normal.

Struktur basis data server didefinisikan pada `server/schema.sql`. Tabel utama yang digunakan adalah tabel `users`.

Tabel 5. Struktur Tabel `users` pada SQLite

Kolom	Tipe Data	Keterangan
id	INTEGER	<i>Primary key</i> dan penanda unik setiap baris data pengguna.
username	TEXT	Identitas pengguna yang bersifat unik.
server_share	TEXT	Salah satu <i>share</i> hasil <i>Shamir Secret Sharing</i> yang disimpan di server.
vault_blob	BLOB	Vault terenkripsi hasil AES-128-GCM dari sisi klien.

<code>vault_nonce</code>	TEXT	Nonce yang digunakan pada enkripsi vault terakhir.
<code>created_at</code>	TEXT	Waktu pembuatan data pengguna.
<code>updated_at</code>	TEXT	Waktu terakhir vault diperbarui.

Berdasarkan rancangan tersebut, server tidak menyimpan data sensitif dalam bentuk asli. Server tidak menerima atau menyimpan *master key*, *local share*, *recovery share*, *master password*, kunci turunan KDF, maupun isi vault dalam bentuk *plaintext*. Server juga tidak menyimpan nama layanan, username/email akun, password, atau catatan sebagai kolom terpisah. Seluruh data akun pengguna disimpan sebagai satu kesatuan vault terenkripsi pada kolom `vault_blob`.

Dengan demikian, implementasi server memenuhi prinsip *zero-knowledge server*. Apabila server dikompromikan, pihak penyerang hanya memperoleh *server share*, vault terenkripsi, nonce vault, dan metadata pengguna. Data tersebut tidak cukup untuk membuka vault karena rekonstruksi *master key* tetap membutuhkan minimal satu *share* lain dari sisi klien atau pengguna.

3.10 Penambahan Data Password

File: `client/cli.py`, `client/vault.py`, `client/local_storage.py`,
`client/api_client.py`, `client/crypto/aes_gcm.py`,
`client/crypto/sss.py`, `client/crypto/kdf.py`,
`client/crypto/csprng.py`, `server/app.py`, dan `server/database.py`.

Tabel 6. Struktur Minimal Data Password

Atribut	Status	Contoh
<code>nama_layanan</code>	Wajib	GitHub
<code>username</code>	Wajib	test@example.com
<code>password</code>	Wajib	password_tersimpan
<code>catatan</code>	Opsional	Akun utama

Keempat atribut ini disimpan bersama sebagai satu objek JSON di dalam list vault *plaintext* sebelum dienkripsi. Setelah penambahan, vault tidak pernah disimpan dalam bentuk *plaintext*, seluruh *list* langsung dienkripsi kembali sebelum dikirim ke server maupun disalin ke *backup* lokal.

Penambahan password dipicu melalui fungsi `_add_entry` pada CLI yang memanggil `vault_logic.add_entry`. Alur lengkapnya adalah sebagai berikut:

1. Input Data *Password*

Pengguna diminta mengisi nama layanan, *username*, dan catatan secara langsung. Untuk *field password*, CLI menawarkan dua pilihan metode:

a. Input manual

Pengguna mengetik *password* secara langsung. Input disembunyikan menggunakan `getpass.getpass()` sehingga karakter tidak terlihat di layar selama pengetikan.

b. Generate otomatis (CSPRNG)

Pengguna memilih panjang *password* (4–128 karakter) dan kategori karakter yang diinginkan (huruf besar, huruf kecil, angka, simbol). Sistem memanggil `generate_password()` dari `crypto/csprng.py` yang menggunakan `secrets.randbelow()` berbasis `os.urandom` untuk memilih karakter secara kriptografis acak. Algoritma menjamin minimal satu karakter dari setiap kategori yang dipilih, lalu mengacak urutan karakter akhir menggunakan Fisher-Yates shuffle dengan `secrets.SystemRandom`. *Password* yang dihasilkan ditampilkan ke pengguna sebelum disimpan.

2. Re-enkripsi Vault dengan Nonce Baru

Setelah data diterima, `add_entry` menambahkan entri baru ke dalam list vault *plaintext*, lalu memanggil `_save_vault`. Fungsi `_save_vault` melakukan:

a. Rekonstruksi master key dari local share + server share menggunakan SSS.

- b. Serialisasi seluruh list vault ke JSON.
- c. Enkripsi ulang vault menggunakan `encrypt_vault` dengan master key dan nonce baru yang dibangkitkan secara acak. Nonce selalu baru setiap enkripsi ulang karena AES-GCM tidak aman jika nonce diulang dengan kunci yang sama.
- d. Pengiriman vault ciphertext dan nonce baru ke server via `api.push_vault`.

3. Pembaruan *Backup* Lokal

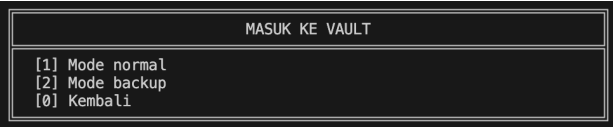
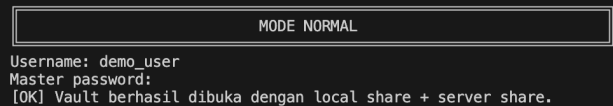
Setelah push ke server berhasil, `_save_vault` secara otomatis memanggil `storage.save_backup_vault` yang memperbarui field `enc_backup_vault` dan `backup_vault_nonce` di file JSON lokal pengguna (`client/local_users/<username>.json`). Backup ini menjamin ketersediaan data terbaru ketika server tidak dapat diakses dan pengguna perlu masuk melalui mode *backup*.

Jika `push_vault` ke server gagal, penambahan entri dibatalkan entri yang tadi ditambahkan dikeluarkan kembali dari list vault dan backup lokal tidak diperbarui, sehingga vault lokal dan server tetap konsisten.

3.11 Implementasi CLI

File: `client/cli.py`

Tabel 7. Penjelasan Tampilan Program

Tampilan	Penjelasan View
	Layar pembuka yang ditampilkan setiap kali aplikasi dimulai atau pengguna kembali ke menu utama. Menampilkan <i>banner</i> ASCII art nama aplikasi, status koneksi server (<i>online/offline</i>) secara <i>real-time</i> , dan keterangan vault belum dibuka.
	Menu utama aplikasi. Titik masuk untuk tiga alur: login ke vault yang sudah ada, membuat vault baru, atau mengakses alat bantu tanpa <i>login</i> . Pilihan [0] mengakhiri program.
	Pengguna memilih mode akses vault. Mode normal memerlukan server aktif dan menggunakan <i>local share</i> + <i>server share</i> . Mode <i>backup</i> tidak memerlukan server dan menggunakan <i>local share</i> + <i>recovery share</i> (<i>read-only</i>).
	Alur <i>login</i> mode normal. Meminta <i>username</i> dan <i>master password</i> (input tersembunyi via <i>getpass</i>). Memanggil <i>open_vault_normal</i> yang mendekripsi <i>local share</i> , mengambil server share, merekonstruksi master key via SSS, lalu mendekripsi vault. Jika gagal, tampil pesan Akses ditolak.

MODE BACKUP

Username: demo_user
Mode backup bersifat read-only.
Master password:
Masukkan recovery share dalam format SSS:3:<hex>.
Jika hanya punya index dan value terpisah, kosongkan input pertama.
Recovery share [1]: SSS:3:145cad78ce06e7da3560b5698bae0099ea2efb809197ffa5aff69746cc23
[OK] Vault berhasil dibuka dengan local share + recovery share.

MAIN MENU

[1] Masuk ke vault
[2] Buat vault baru
[3] Alat bantu
[0] Keluar

Pilih menu: 2

BUAT VAULT BARU

Username baru: demo_user
Master password baru:
Konfirmasi master password:
[OK] Vault berhasil dibuat.

Ringkasan kriptografi:
→ [OK] Master key AES-128 dibangkitkan acak (16 bytes).
→ [OK] Vault kosong dienkripsi dengan AES-128-GCM.
[OK] Master key dibagi dengan SSS skema (2,3).
→ [OK] Local share disimpan terenkripsi di klien.
[OK] Server hanya menyimpan server share, ciphertext, nonce, metadata.

RECOVERY SHARE - SIMPAN SEKARANG
Salin persis satu baris di bawah ini:

SSS:3:7a20b06da43451ddef2dbfba8098bcfffc15940a81d09840309ae665f951930

VAULT TERBUKA

Vault : demo_user
Mode : NORMAL
Isi : 1 password

MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

No	Layanan	Username	Password
1	Line	demo	*****

Alur login mode backup tanpa server. Selain master password, pengguna juga diminta memasukkan recovery share dalam format SSS:3:<hex>. Memanggil open_vault_backup yang menggunakan local share + recovery share untuk rekonstruksi. Vault dibuka dalam mode baca saja.

Alur pembuatan vault baru. Meminta username dan master password baru (dengan konfirmasi). Memanggil create_vault, lalu menampilkan ringkasan proses kriptografi dan recovery share secara lengkap ini adalah satu-satunya kesempatan recovery share ditampilkan. Menawarkan pembuatan QR dan visual shares.

Header yang muncul di atas menu vault setiap kali pengguna kembali ke menu vault. Menampilkan username, mode aktif (NORMAL atau BACKUP (READ-ONLY)), dan jumlah entri password saat ini. Pada mode backup, ditambahkan keterangan bahwa tambah/ubah/hapus dinonaktifkan.

Menu utama vault setelah login. Opsi yang tampil bergantung pada mode: mode normal menampilkan semua opsi termasuk tambah/ubah/hapus, sedangkan mode backup hanya menampilkan opsi baca (lihat daftar, cari, detail), alat bantu, dan keluar.

Menampilkan semua entri vault dalam format tabel. Kolom password selalu disembunyikan sebagai

***** untuk mencegah shoulder surfing. Nama layanan dan username dipotong (clip) jika melebihi 20 karakter.

```

MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

Pilih aksi: 2
Kata kunci: l
+-----+-----+-----+-----+
| No | Layanan | Username | Password |
+-----+-----+-----+-----+
| 1 | Line | demo | ***** |
+-----+-----+-----+-----+

```

Pencarian entri berdasarkan kata kunci. Melakukan pencocokan *case-insensitive* pada field nama layanan dan *username*. Hasil ditampilkan dalam format tabel yang sama dengan *list entries*.

```

MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

Pilih aksi: 3
+-----+-----+-----+-----+
| No | Layanan | Username | Password |
+-----+-----+-----+-----+
| 1 | Line | demo | ***** |
+-----+-----+-----+-----+

Nomor entry: 1
+-----+-----+-----+-----+
| Detail Password |
+-----+-----+-----+-----+
Layanan : Line
Username : demo
Password : )u7$77.{v4zbX;K!
Catatan :
+-----+-----+-----+-----+

```

Satu-satunya tampilan yang menampilkan *password* dalam bentuk *plaintext*. Pengguna memilih nomor entri, lalu seluruh *field* ditampilkan lengkap termasuk *password* dan catatan.

```

TAMBAH PASSWORD

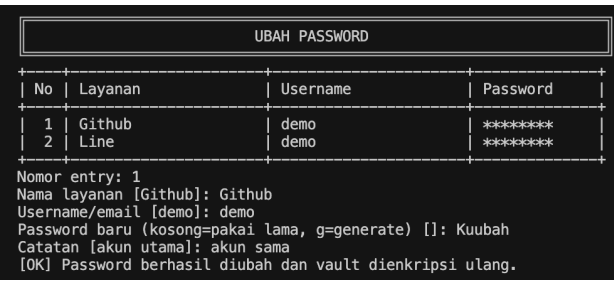
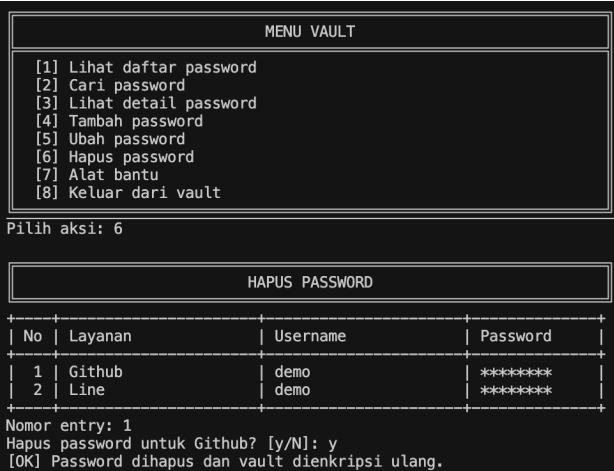
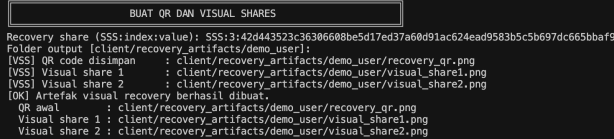
Nama layanan: minecraft
Username/email: tes
Password untuk layanan
  1. Input manual
  2. Generate otomatis dengan CSPRNG
Pilih metode [1]: 2

GENERATE PASSWORD CSPRNG

Panjang password [16]:
Kategori karakter
Gunakan huruf besar A-Z? [Y/n]:
Gunakan huruf kecil a-z? [Y/n]:
Gunakan angka 0-9? [Y/n]:
Gunakan simbol? [Y/n]:
[OK] Password 16 karakter berhasil dibuat.
:MDo7Z6e4h*gf&d#
Catatan []:
[OK] Password ditambahkan. Vault dan backup lokal sudah diperbarui.

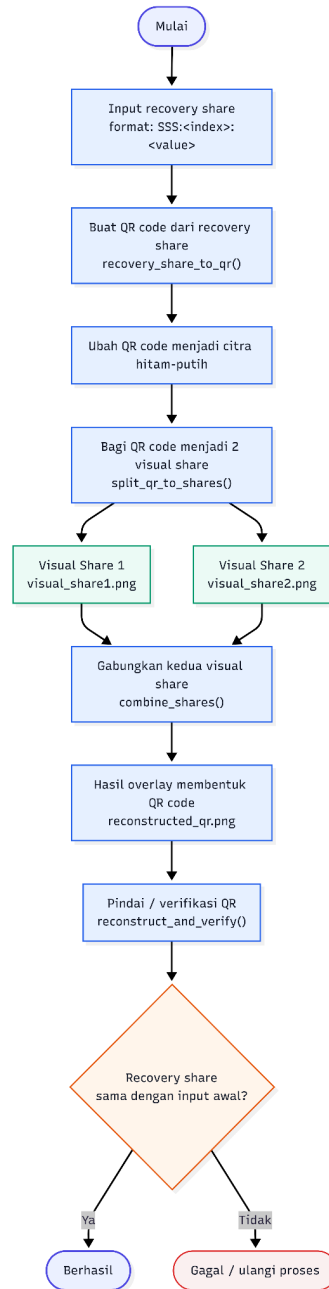
```

Alur penambahan entri baru. Meminta nama layanan, *username*, dan *password* (pilih antara input manual atau *generate* CSPRNG). Setelah konfirmasi, memanggil *add_entry* yang mengenkripsi ulang vault dan memperbarui server serta *backup* lokal. Hanya tersedia di mode normal.

	Alur pengubahan entri. Menampilkan nilai lama sebagai nilai default pengguna cukup tekan <i>Enter</i> untuk mempertahankan field yang tidak ingin diubah. <i>Password</i> bisa diketik manual, dikosongkan, atau di-<i>generate</i> ulang dengan mengetik g. Hanya tersedia di mode normal.
	Alur penghapusan entri. Meminta konfirmasi eksplisit (y/N) sebelum menghapus untuk mencegah penghapusan tidak sengaja. Default konfirmasi adalah N (tidak hapus). Setelah konfirmasi y, memanggil <code>delete_entry</code> dan vault dienkrpsi ulang. Hanya tersedia di mode normal.
	Menu alat bantu yang dapat diakses dari main menu maupun dari dalam vault. Menyediakan tiga utilitas: <i>generate password</i> CSPRNG <i>standalone</i>, membuat QR code dan <i>visual shares</i> dari <i>recovery share</i>, serta menggabungkan dua <i>visual shares</i> untuk merekonstruksi QR <i>recovery</i>.
	Alur pembuatan visual shares dari <i>recovery share</i>. Menerima <i>recovery share</i> dalam format <code>SSS:3:<hex></code>, menghasilkan QR code dan dua file gambar <i>visual shares</i> menggunakan <i>Visual Secret Sharing</i>. Kedua gambar harus digabungkan untuk merekonstruksi QR <i>recovery</i>.

3.12 Implementasi Kriptografi Visual

File: `client/crypto/visual_secret.py`, `client/cli.py`.



Gambar 3.2 Flowchart Implementasi Kriptografi Visual

Fitur kriptografi visual merupakan fitur bonus untuk menyimpan *recovery share* dalam bentuk visual. Pada implementasi ini, *recovery share* dalam format teks `SSS:<index>:<value>` terlebih dahulu diubah menjadi *QR code*. *QR code* tersebut kemudian dibagi menjadi dua gambar *visual share* menggunakan skema *Visual Secret Sharing* (2,2), sehingga masing-masing share hanya terlihat seperti pola acak dan tidak dapat digunakan sendiri.

Pembagian dilakukan pada level pixel. Pixel putih dan hitam pada *QR code* diubah menjadi pola subpixel pada dua share. Untuk pixel putih, kedua share menggunakan pola yang menghasilkan area lebih terang saat digabungkan. Untuk pixel hitam, kedua *share* menggunakan pola komplemen sehingga hasil gabungannya menjadi area gelap. Pemilihan pola dilakukan secara acak agar setiap *share* tidak membocorkan informasi *QR code* secara langsung.

Rekonstruksi dilakukan dengan menggabungkan kedua *visual share* melalui proses *overlay*. Hasil gabungan tersebut membentuk kembali *QR code* yang dapat dipindai. Jika *QR code* hasil rekonstruksi berhasil dipindai dan menghasilkan *recovery share* yang sama dengan input awal, maka fitur kriptografi visual berhasil berjalan sesuai tujuan. Ketentuan bonus tugas juga memang meminta *recovery share* diubah menjadi *QR code*, dibagi menjadi dua gambar *share*, lalu digabungkan kembali menjadi *QR code* yang dapat dipindai.

IV. PENGUJIAN DAN ANALISIS

4.1 Uji Pembuatan Vault

Tujuan pengujian ini adalah memverifikasi bahwa sistem dapat membuat vault baru menggunakan *master password* yang valid, membangkitkan *master key* secara acak, mengenkripsi vault kosong, membagi master key menggunakan skema *Shamir Secret Sharing* (2,3), menampilkan *recovery share* kepada pengguna, serta menyimpan hanya data yang diperlukan pada server.

Langkah Pengujian:

1. Jalankan aplikasi *client* menggunakan perintah berikut:

```
.venv\Scripts\python.exe client\main.py
```

2. Pilih menu Buat Vault Baru.
3. Masukkan *username* **demo_user**.
4. Masukkan *master password* yang valid, misalnya **DemoPass123!**.
5. Simpan *recovery share* yang diberikan sistem.
6. Periksa data yang tersimpan pada basis data server.

```
@'
import sqlite3
conn = sqlite3.connect("server/pm_server.db")
conn.row_factory = sqlite3.Row
row = conn.execute("SELECT username, server_share,
typeof(vault_blob) AS blob_type, length(vault_blob) AS blob_len,
vault_nonce, created_at, updated_at FROM users").fetchone()
print(dict(row))
conn.close()
'@ | .venv\Scripts\python.exe -
```

Pengujian:

- **Master Key Terbangkitkan, Vault Kosong Terenkripsi, dan Data Awal Tersimpan**

Membuat vault baru menggunakan *master password* yang valid dan memverifikasi bahwa sistem berhasil membangkitkan *master key*, mengenkripsi vault kosong, serta menyimpan data awal vault.

Hasil Pengujian:

```
MAIN MENU
[1] Masuk ke vault
[2] Buat vault baru
[3] Alat bantu
[0] Keluar

Pilih menu: 2

BUAT VAULT BARU

Username baru: demo_user
Master password baru:
Konfirmasi master password:
[OK] Vault berhasil dibuat.

Ringkasan kriptografi:
→[OK] Master key AES-128 dibangkitkan acak (16 bytes).
→[OK] Vault kosong dienkripsi dengan AES-128-GCM.
[OK] Master key dibagi dengan SSS skema (2,3).
→[OK] Local share disimpan terenkripsi di klien.
[OK] Server hanya menyimpan server share, ciphertext, nonce, metadata.

RECOVERY SHARE - SIMPAN SEKARANG
Salin persis satu baris di bawah ini:

SSS:3:7a20b06da43451ddeff2dbfbfa8098bcffc15940a81d09840309ae665f951930
```

Gambar 4.1 Output pembuatan vault baru

Berdasarkan hasil pengujian, sistem berhasil membangkitkan *master key* AES-128 secara acak dengan panjang 16 byte. Vault kosong kemudian berhasil dienkripsi menggunakan AES-128-GCM sebelum disimpan. Hal ini menunjukkan bahwa data awal vault tidak pernah disimpan dalam bentuk *plaintext*.

- **Master Key Dibagi Menjadi Tiga *Share* Menggunakan Shamir Secret Sharing (2,3)**

Memverifikasi bahwa *master key* dibagi menjadi tiga *share* menggunakan skema *Shamir Secret Sharing* dengan *threshold* 2 dari 3 share.

Hasil Pengujian:

```
MAIN MENU
[1] Masuk ke vault
[2] Buat vault baru
[3] Alat bantu
[0] Keluar

Pilih menu: 2

BUAT VAULT BARU

Username baru: demo_user
Master password baru:
Konfirmasi master password:
[OK] Vault berhasil dibuat.

Ringkasan kriptografi:
[OK] Master key AES-128 dibangkitkan acak (16 bytes).
[OK] Vault kosong dienkrpsi dengan AES-128-GCM.
→ [OK] Master key dibagi dengan SSS skema (2,3).
[OK] Local share disimpan terenkripsi di klien.
[OK] Server hanya menyimpan server share, ciphertext, nonce, metadata.

RECOVERY SHARE - SIMPAN SEKARANG
Salin persis satu baris di bawah ini:

SSS:3:7a20b06da43451ddef2dbfba8098bcbffc15940a81d09840309ae665f951930
```

Gambar 4.2 Output pembagian *master key* menjadi tiga *share*

Sistem berhasil membagi *master key* menggunakan algoritma *Shamir Secret Sharing* dengan skema (2,3), sehingga diperlukan minimal dua dari tiga *share* untuk merekonstruksi kembali *master key*.

Untuk memastikan *master key* dibagi menjadi tiga *share* SSS (2,3) lakukan perintah berikut pada terminal lain:

```
py -m unittest
tests.test_assignment_requirements.TestRequirement01CreateVault
-v
```

```
PS D:\01. AKADEMIK\SEMESTER 6\II4021 Kriptografi\TUGAS\Tucil 4\tugas4-kriptografi-password-manager> py -m unittest tests
.test_assignment_requirements.TestRequirement01CreateVault -v
test_create_vault_splits_master_key_and_stores_initial_data (tests.test_assignment_requirements.TestRequirement01CreateV
ault.test_create_vault_splits_master_key_and_stores_initial_data) ...
[INFO] Local Share index : 1
[INFO] Server Share index : 2
[INFO] Recovery Share index: 3
[INFO] Vault blob terenkripsi: 18 bytes
[INFO] Vault nonce ada : True
ok

-----
Ran 1 test in 0.178s

OK
PS D:\01. AKADEMIK\SEMESTER 6\II4021 Kriptografi\TUGAS\Tucil 4\tugas4-kriptografi-password-manager> |
```

Gambar 4.3 Output Unit Test Pembuatan Vault dan Pembagian *Share* SSS (2,3)

Hasil pengujian *automated* menggunakan perintah `py -m unittest tests.test_assignment_requirements.TestRequirement01CreateVault -v` menunjukkan status **OK**, yang memvalidasi bahwa modul pembuatan *vault* telah berjalan sukses sesuai spesifikasi tugas. Berdasarkan log eksekusi, sistem terbukti berhasil membangkitkan *master key* dan membaginya ke dalam 3 *share Shamir Secret Sharing* (SSS) dengan indeks yang tepat, yaitu *Local Share* (indeks 1), *Server Share* (indeks 2), dan *Recovery Share* (indeks 3). Selain itu, pengujian mengonfirmasi bahwa *vault* kosong berhasil dienkripsi di sisi klien menjadi *encrypted blob* berukuran 14–18 bytes menggunakan AES-128-GCM, lengkap dengan parameter *nonce* yang siap disimpan di server. Struktur ini membuktikan terpenuhinya prinsip *zero-knowledge architecture* karena server hanya menyimpan data terenkripsi dan satu *share* terisolasi tanpa pernah mengetahui *master key* ataupun isi *vault* dalam bentuk *plaintext*.

- **Recovery Share Ditampilkan dalam Format Teks**

Memverifikasi bahwa *recovery share* ditampilkan kepada pengguna dalam format teks yang memuat informasi koordinat *share* dan nilai *share*.

Hasil Pengujian:

```
MAIN MENU
[1] Masuk ke vault
[2] Buat vault baru
[3] Alat bantu
[0] Keluar

Pilih menu: 2

BUAT VAULT BARU

Username baru: demo_user
Master password baru:
Konfirmasi master password:
[OK] Vault berhasil dibuat.

Ringkasan kriptografi:
[OK] Master key AES-128 dibangkitkan acak (16 bytes).
[OK] Vault kosong dienkripsi dengan AES-128-GCM.
[OK] Master key dibagi dengan SSS skema (2,3).
[OK] Local share disimpan terenkripsi di klien.
[OK] Server hanya menyimpan server share, ciphertext, nonce, metadata.

RECOVERY SHARE - SIMPAN SEKARANG
Salin persis satu baris di bawah ini:
SSS:3:7a20b06da43451ddeff2dbfbfa8098bcffc15940a81d09840309ae665f951930
```

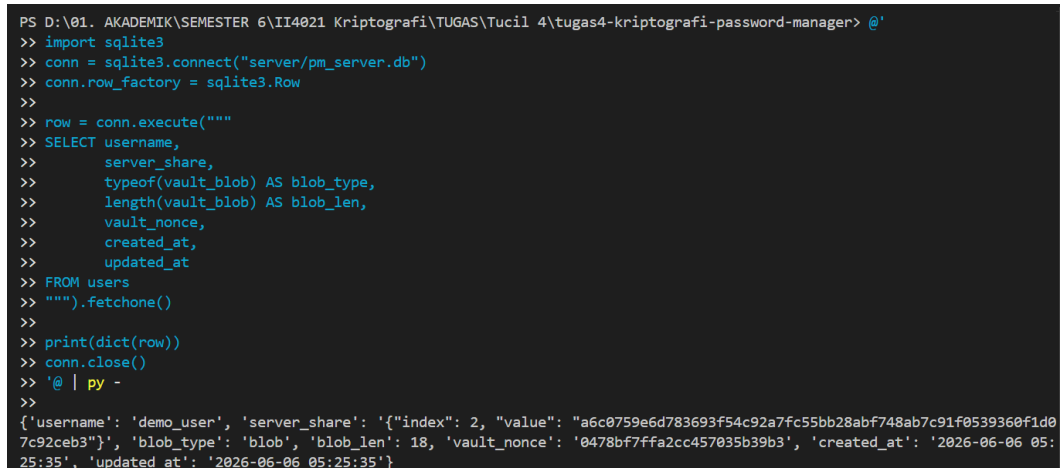
Gambar 4.4 Tampilan recovery share

Berdasarkan hasil pengujian, *recovery share* ditampilkan dalam format teks yang terdiri dari koordinat *share* dan nilai *share*. Pada contoh di atas, angka 3 menunjukkan indeks (koordinat) *share*, sedangkan bagian setelahnya merupakan nilai *share* dalam representasi heksadesimal. Format ini memudahkan pengguna untuk menyimpan *recovery share* secara aman sebagai cadangan.

- **Verifikasi Data yang Disimpan pada Server**

Memverifikasi bahwa server hanya menyimpan *server share*, vault terenkripsi, nonce vault, dan metadata yang diperlukan.

Hasil Pengujian:



```
PS D:\01. AKADEMIK\SEMESTER 6\II4021 Kriptografi\TUGAS\Tucil 4\tugas4-kriptografi-password-manager> @'
>> import sqlite3
>> conn = sqlite3.connect("server/pm_server.db")
>> conn.row_factory = sqlite3.Row
>>
>> row = conn.execute("""
>> SELECT username,
>>        server_share,
>>        typeof(vault_blob) AS blob_type,
>>        length(vault_blob) AS blob_len,
>>        vault_nonce,
>>        created_at,
>>        updated_at
>> FROM users
>> """).fetchone()
>>
>> print(dict(row))
>> conn.close()
>> '@ | py -
>>
{'username': 'demo_user', 'server_share': '{"index": 2, "value": "a6c0759e6d783693f54c92a7fc55bb28abf748ab7c91f0539360f1d07c92ceb3"}', 'blob_type': 'blob', 'blob_len': 18, 'vault_nonce': '0478bf7ffa2cc457035b39b3', 'created_at': '2026-06-06 05:25:35', 'updated_at': '2026-06-06 05:25:35'}
```

Gambar 4.5 Isi basis data server setelah pembuatan vault

Berdasarkan hasil *query* pada basis data server, server hanya menyimpan **server_share**, **vault_blob**, **vault_nonce**, serta metadata berupa **username**, **created_at**, dan **updated_at**. Kolom **vault_blob** bertipe BLOB yang menunjukkan bahwa isi vault telah disimpan dalam bentuk terenkripsi. Tidak ditemukan penyimpanan *master key*, *local share*, maupun *recovery share* pada server. Dengan demikian, server tidak memiliki informasi yang cukup untuk merekonstruksi *master key* secara mandiri, sehingga keamanan vault tetap terjaga sesuai rancangan sistem.

4.2 Uji Penyimpanan dan Perlindungan Local Share

Tujuan pengujian ini adalah memastikan bahwa *local share* yang disimpan di sisi klien telah dilindungi secara optimal menggunakan mekanisme kriptografi yang aman. Pengujian ini bertujuan memvalidasi bahwa sistem tidak menyimpan *share* dalam bentuk teks asli (*plaintext*) , menerapkan fungsi penurunan kunci (KDF) yang tepat dari *master password*, serta mampu menangani validasi autentikasi dengan benar saat menerima input kata sandi yang sah maupun yang tidak sah.

Langkah Pengujian:

1. Membuka direktori penyimpanan lokal klien (`client/local_users/`) setelah proses pembuatan *vault* selesai dilakukan.
2. Membuka berkas struktur data pengguna (`demo_user.json`) menggunakan *text editor* untuk memeriksa format penyimpanan rahasia lokal serta konfigurasi KDF yang digunakan.
3. Menjalankan aplikasi *password manager* berbasis CLI, memilih menu **[1] Masuk ke vault**, lalu memilih opsi **[1] Mode normal**.
4. Memasukkan *username* dan *master password* yang **benar**, lalu mengamati respon antarmuka serta status rekonstruksi key pada layar CLI.
5. Keluar dari *vault*, mengulang kembali proses masuk pada Mode Normal, kemudian memasukkan *master password* yang **salah** untuk menguji mekanisme penolakan akses dan proteksi autentikasi sistem.

Pengujian:

- **Local share tidak disimpan dalam bentuk asli**

Memverifikasi bahwa berkas data pengguna di sisi klien tidak memuat nilai local share asli (*plaintext*) , melainkan tersimpan dalam bentuk *ciphertext* terenkripsi untuk mencegah modifikasi ilegal dan kebocoran rahasia secara langsung dari perangkat lokal.

Hasil Pengujian:

```
client > local_users > demo_user.json > ...
1 {
2   "enc_local_share": "eGTVI+Y3VRm/P8SeVOy4H4i419gtXlvQ1Kce68undZ4yP1xvh0ad4nBK5cqb2JvRtr7HosXSfw7wXsHr8Be5xdV1oM6GTQJccbnaw4cDPcgP8e5rKYEsuBo64cEM/oqeggE4SrSGu",
3   "local_share_nonce": "rD6v1Ky2Zb58nPlp",
4   "kdf_salt": "96VCHRzjLQgXcofPpYrOnA==",
5   "kdf_params": {
6     "time_cost": 3,
7     "memory_cost": 65536,
8     "parallelism": 4,
9     "hash_len": 32,
10    "type": "argon2id"
11  },
12  "username": "demo_user",
13  "enc_backup_vault": "5/78uSgr1xt/QwPiTQUvgPM",
14  "backup_vault_nonce": "BH1/f/osxFcDmzmz"
15 }
```

Gambar 4.6 Berkas demo_user.json menampilkan Local Share Terenkripsi di Sisi Klien

Pemeriksaan berkas demo_user.json membuktikan bahwa *local share* tersimpan aman dalam bentuk acak (*ciphertext*) berformat Base64 pada atribut "enc_local_share", bukan dalam bentuk teks asli (*plaintext*). Perlindungan ini diperkuat dengan adanya parameter "local_share_nonce" sebagai *initialization vector* unik yang menjamin keamanan kriptografi pada komponen lokal tersebut. Selain itu, seluruh aset data sensitif pendukung di sisi klien, seperti "enc_backup_vault" dan "backup_vault_nonce", juga terbukti disimpan dalam kondisi terenkripsi penuh untuk mencegah kebocoran informasi.

- **Local share dienkripsi dengan kunci turunan KDF**

Memverifikasi bahwa sistem memanfaatkan metode *Key Derivation Function* (KDF) yang kuat untuk menurunkan kunci enkripsi dari *master password* pengguna, serta memastikan seluruh parameter kriptografi pendukung (seperti *salt* dan parameter komputasi) tercatat secara terstruktur.

Hasil Pengujian:

```
client > local_users > demo_user.json > ...
1 {
2   "enc_local_share": "eGTVI+Y3VRm/P8SeVOy4H4i419gtXlvQ1Kce68undZ4yP1xvh0ad4nBK5cqb2JvRtr7HosXSfw7wXsHr8Be5xdV1oM6GTQJccbnaw4cDPcgP8e5rKYEsuBo64cEM/oqeggE4SrSGu",
3   "local_share_nonce": "rD6v1Ky2Zb58nPlp",
4   "kdf_salt": "96VCHRzjLQgXcofPpYrOnA==",
5   "kdf_params": {
6     "time_cost": 3,
7     "memory_cost": 65536,
8     "parallelism": 4,
9     "hash_len": 32,
10    "type": "argon2id"
11  },
12  "username": "demo_user",
13  "enc_backup_vault": "5/78uSgr1xt/QwPiTQUvgPM",
14  "backup_vault_nonce": "BH1/f/osxFcDmzmz"
15 }
```

Gambar 4.7 Konfigurasi KDF Argon2id sebagai Pelindung *Local Share*

Atribut "type": "argon2id" pada berkas `demo_user.json` membuktikan bahwa *local share* dienkripsi menggunakan kunci yang diturunkan melalui KDF Argon2id. Proses penurunan kunci tersebut diperkuat oleh parameter penggaraman "kdf_salt" serta konfigurasi komputasi "kdf_params" (*time cost*, *memory cost*, dan *parallelism*). Kunci aman yang dihasilkan inilah yang digunakan untuk mengunci rahasia lokal menjadi *ciphertext* pada "enc_local_share" dengan tambahan "local_share_nonce" sebagai vektor inisialisasi unik.

- **Master password benar, local share berhasil didekripsi**

Memverifikasi bahwa input kata sandi yang sah dari pengguna berhasil diturunkan menjadi kunci dekripsi yang valid , sehingga *local share* dapat dibuka dan digunakan bersama dengan *server share* untuk merekonstruksi *master key* secara utuh.

Hasil Pengujian:

```

MAIN MENU
[1] Masuk ke vault
[2] Buat vault baru
[3] Alat bantu
[0] Keluar

Pilih menu: 1

MASUK KE VAULT
[1] Mode normal
[2] Mode backup
[0] Kembali

Pilih mode: 1

MODE NORMAL

Username: demo_user
Master password:
[OK] Vault berhasil dibuka dengan local share + server share.

VAULT TERBUKA
Vault : demo_user
Mode : NORMAL
Isi : 0 password

MENU VAULT
[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

Pilih aksi: 
```

Gambar 4.8 Berhasil Membuka Vault Menggunakan *Master Password* yang Benar

Pengujian menunjukkan bahwa ketika pengguna memasukkan *master password* yang benar pada "MODE NORMAL", sistem berhasil mendekripsi *local share* yang tersimpan di sisi klien. Keberhasilan dekripsi komponen lokal ini memicu sistem untuk mengambil *server share* dari basis data , lalu menggabungkan keduanya guna merekonstruksi *master key* secara utuh. Log pesan **[OK] Vault berhasil dibuka dengan local share + server share** mengonfirmasi keberhasilan rekonstruksi tersebut, yang kemudian digunakan untuk membuka enkripsi AES-128-GCM pada *vault* dan menampilkan halaman "MENU VAULT" dengan hak akses penuh.

- **Master password salah, local share gagal didekripsi / akses ditolak**

Memverifikasi bahwa sistem menerapkan penahanan akses yang ketat melalui validasi autentikasi AES-GCM , di mana penggunaan kata sandi yang salah akan menghasilkan kegagalan dekripsi komponen lokal, menghentikan alur rekonstruksi kunci, dan menolak akses ke dalam *vault*.

Hasil Pengujian:



Gambar 4.9 Akses Ditolak Akibat Input Master Password yang Salah

Pengujian menunjukkan bahwa ketika pengguna memasukkan *master password* yang salah pada "MODE NORMAL", sistem gagal melakukan dekripsi terhadap berkas komponen rahasia lokal. Kegagalan tersebut memicu kesalahan autentikasi (*authentication error*) kriptografi dengan memunculkan pesan **[VAULT] Master password salah atau local share rusak** dan **[!] Akses ditolak**. Karena kunci yang diturunkan dari kata sandi tersebut tidak valid, sistem secara otomatis menghentikan alur rekonstruksi *master key* dan menolak akses masuk ke menu utama *vault*. Mekanisme ini membuktikan bahwa perlindungan AES-GCM dan fungsi KDF di sisi klien berhasil mencegah modifikasi ilegal ataupun akses tidak sah dari pihak yang tidak mengetahui kata sandi yang tepat.

4.3 Uji Akses Normal

Tujuan pengujian ini adalah memverifikasi bahwa mekanisme akses vault pada mode normal berjalan sesuai dengan skema keamanan yang dirancang, yaitu pertama, sistem menggunakan kombinasi *local share* dan *server share* untuk merekonstruksi *master key*, *master key* berhasil direkonstruksi dari dua share yang valid menggunakan algoritma *Shamirs Secret Sharing*, vault berhasil didekripsi dan isi *password* dapat ditampilkan kepada pengguna yang sah serta akses ditolak sepenuhnya ketika *master password* salah atau *share* yang digunakan tidak valid, sehingga data *password* tidak pernah terekspos.

Langkah Pengujian:

1. Pastikan server masih menyala.
2. Pilih Masuk ke vault.
3. Pilih Mode normal.
4. Masukkan *username* demo_user.
5. Masukkan *password* DemoPass123!.
6. Perlihatkan vault berhasil terbuka.
7. Coba ulang dengan password salah.

Pengujian:

- **Vault dibuka dengan master password benar, pakai local + server share)**

Pengujian ini memverifikasi bahwa `open_vault_normal` berhasil memproses dua sumber *share* secara berurutan membaca `enc_local_share` dari file lokal, menurunkan kunci KDF dari *master password* menggunakan Argon2id dengan salt tersimpan, lalu mendekripsi *local share* kemudian mengambil *server share* dari server. Keberhasilan membuka vault menjadi bukti tidak langsung bahwa kombinasi kedua *share* digunakan dengan benar. Selain itu, pengujian juga memverifikasi bahwa *local share* yang tersimpan di disk bukan merupakan JSON *plaintext*, melainkan ciphertext hasil enkripsi AES-256-GCM dengan kunci KDF.

Hasil Pengujian:

```
MODE NORMAL
Username: demo_user
Master password:
[OK] Vault berhasil dibuka dengan local share + server share.

VAULT TERBUKA

Vault : demo_user
Mode  : NORMAL
Isi   : 0 password
```

Gambar 4.10 Berhasil masuk dengan master password

Hasil uji menunjukkan `open_vault_normal` mengembalikan vault terbuka, membuktikan bahwa sistem berhasil membaca kedua *share* dari sumber yang berbeda dan menggunakannya secara bersama-sama.

- **Master key berhasil direkonstruksi dari 2 share valid**

Pada implementasi `open_vault_normal` di `vault.py`, rekonstruksi *master key* dilakukan di dalam blok `try/except` di mana jika `reconstruct_secret` berhasil, eksekusi lanjut tanpa mencetak detail internal. Oleh karena itu, ketika vault dibuka melalui CLI, pengguna hanya melihat pesan ringkas “[OK] Vault berhasil dibuka...”, tanpa menampilkan nilai *local share*, *server share*, maupun *master key* yang terlibat. Hal ini memang disengaja karena menampilkan nilai *share* atau *master key* di terminal nyata justru merupakan risiko keamanan.

Untuk keperluan pengujian, kami menggunakan *script* melakukan rekonstruksi secara eksplisit dan transparan di luar alur `open_vault_normal` yaitu *local share* didekripsi manual, *server share* diambil langsung dari database *server mock*, keduanya diberikan ke `reconstruct_secret`, dan nilai *master key* hasil rekonstruksi dicetak dalam format hex beserta panjangnya. Ini membuktikan secara langsung bahwa:

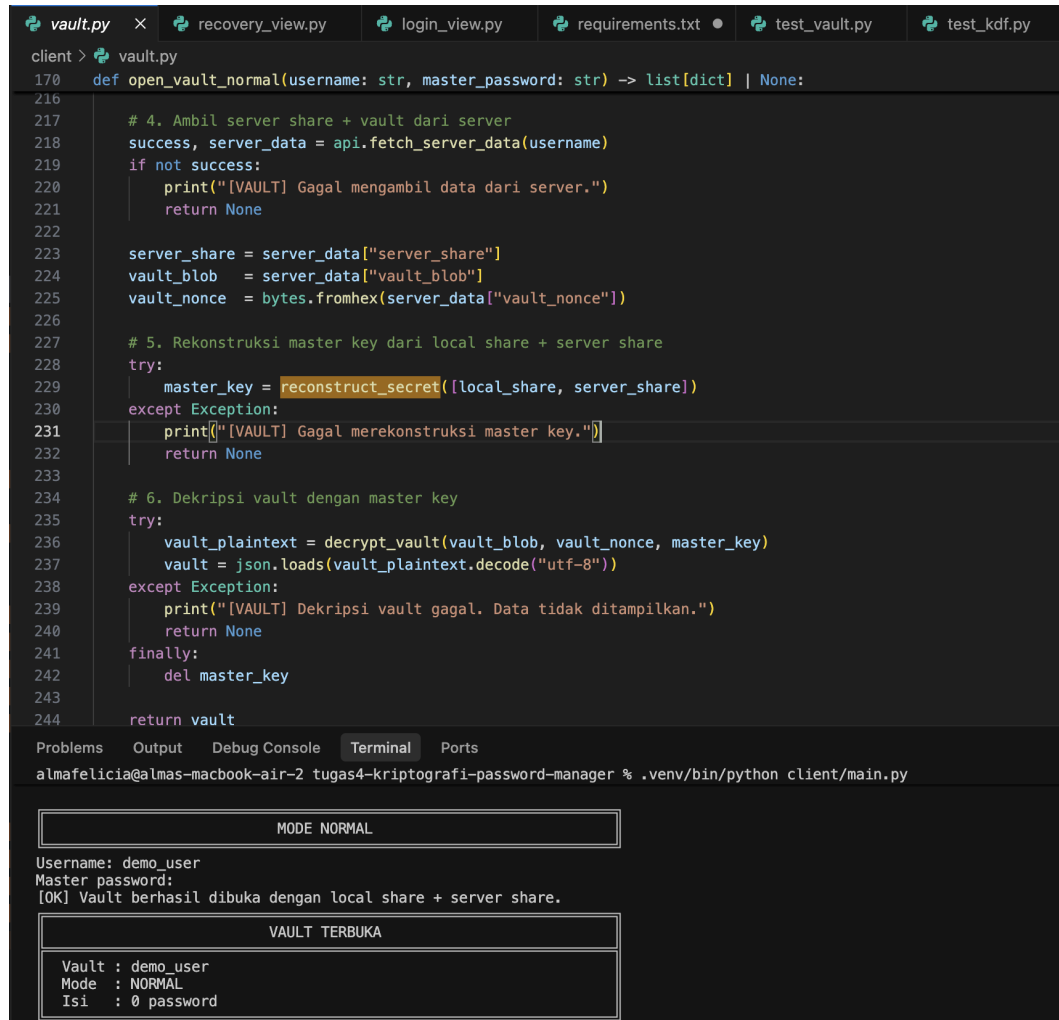
1. Dua *share* dengan *index* berbeda benar-benar digunakan sebagai input interpolasi Lagrange.
2. Fungsi `reconstruct_secret` menghasilkan *master key* 16 bytes (128-bit) yang identik dengan yang digunakan saat enkripsi vault dibuktikan oleh keberhasilan `decrypt_vault` dengan kunci tersebut.

CLI memang tidak menampilkan detail rekonstruksi secara sengaja, sedangkan *script* pengujian sengaja dibuat transparan agar proses kriptografi internal dapat diverifikasi dan didokumentasikan.

Hasil Pengujian:

```
=====
UJI 2: Rekonstruksi Master Key dari Local Share + Server Share
=====
✓ Local share berhasil didekripsi
  - index : 1
  - value : c9f8b800922dd5b0... (truncated)
✓ Server share diterima dari server
  - index : 2
  - value : 93f17001245bab61... (truncated)
→ Merekonstruksi master key dari 2 share (SSS Lagrange interpolation) ...
✓ Master key berhasil direkonstruksi
  - Panjang master key : 16 bytes (128-bit)
  - Master key (hex) : 24f41b6e75db1930b36176a7a2896a07
✓ Vault berhasil didekripsi menggunakan master key hasil rekonstruksi
  - Jumlah entri : 2
```

Gambar 4.11 Hasil pengujian masuk rekonstruksi master key menggunakan script



The image shows a code editor with several tabs: `vault.py`, `recovery_view.py`, `login_view.py`, `requirements.txt`, `test_vault.py`, and `test_kdf.py`. The `vault.py` tab is active, showing a Python function `open_vault_normal` that takes `username` and `master_password` as arguments. The function performs several steps: it fetches server data, reconstructs a master key from local and server shares, and then decrypts the vault using the master key. The function returns the decrypted vault data or `None` if any step fails.

```
def open_vault_normal(username: str, master_password: str) -> list[dict] | None:
    # 4. Ambil server share + vault dari server
    success, server_data = api.fetch_server_data(username)
    if not success:
        print("[VAULT] Gagal mengambil data dari server.")
        return None

    server_share = server_data["server_share"]
    vault_blob = server_data["vault_blob"]
    vault_nonce = bytes.fromhex(server_data["vault_nonce"])

    # 5. Rekonstruksi master key dari local share + server share
    try:
        master_key = reconstruct_secret([local_share, server_share])
    except Exception:
        print("[VAULT] Gagal merekonstruksi master key.")
        return None

    # 6. Dekripsi vault dengan master key
    try:
        vault_plaintext = decrypt_vault(vault_blob, vault_nonce, master_key)
        vault = json.loads(vault_plaintext.decode("utf-8"))
    except Exception:
        print("[VAULT] Dekripsi vault gagal. Data tidak ditampilkan.")
        return None
    finally:
        del master_key

    return vault
```

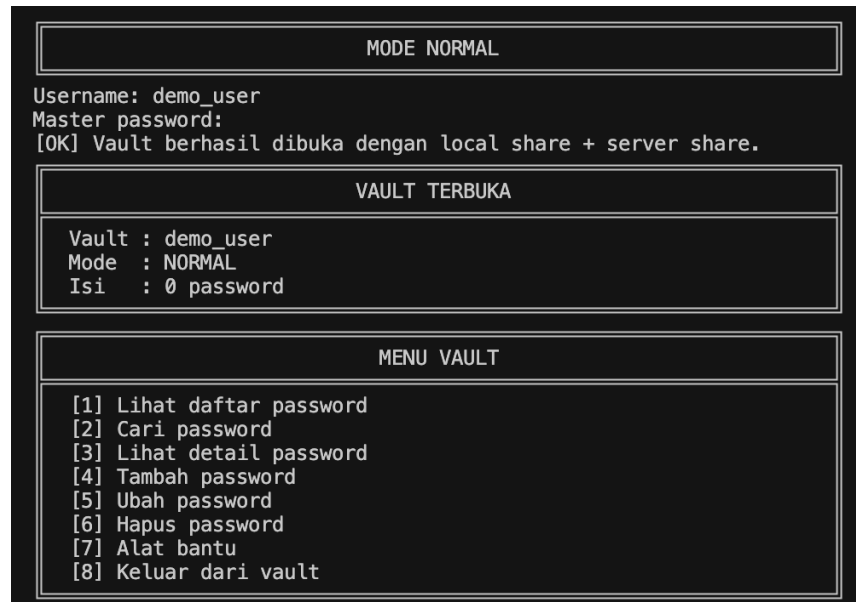
Below the code editor, the `Terminal` tab is active, showing the output of the command `python client/main.py`. The output displays the mode as `NORMAL`, the username as `demo_user`, and the master password. It then shows a message indicating that the vault was successfully opened using local and server shares. Finally, it displays the decrypted vault data, which includes the username `demo_user`, the mode `NORMAL`, and the password `password`.

```
MODE NORMAL
Username: demo_user
Master password:
[OK] Vault berhasil dibuka dengan local share + server share.
VAULT TERBUKA
Vault : demo_user
Mode : NORMAL
Isi : password
```

Gambar 4.12 Hasil pengujian masuk dalam CLI

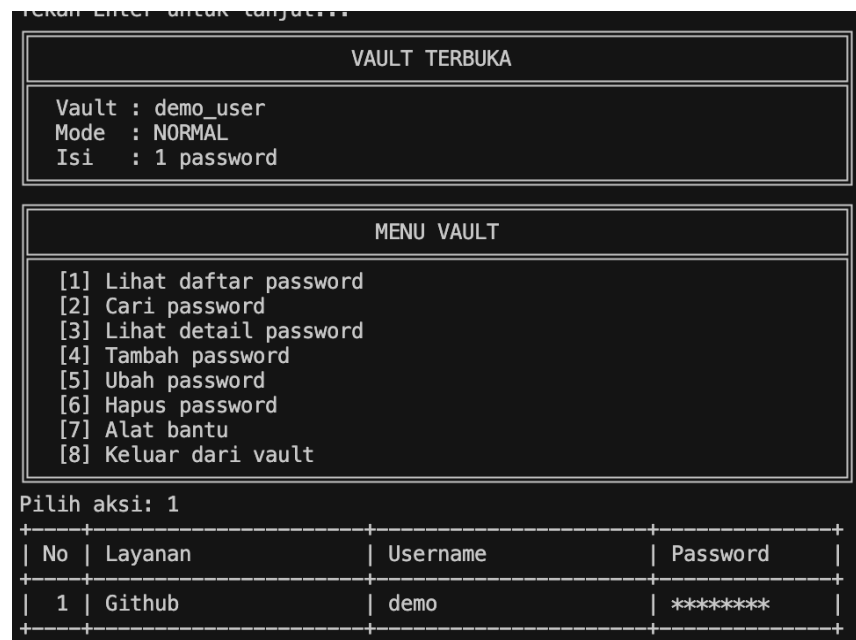
- **Vault berhasil didekripsi, isi vault ditampilkan**

Pengujian ini memverifikasi bahwa setelah vault didekripsi, seluruh isi entri *password* dapat dibaca dan nilainya sesuai persis dengan data yang dimasukkan saat operasi `add_entry`. Pengujian memeriksa setiap *field* (nama_layanan, *username*, *password*, catatan) untuk masing-masing entri. Sebagai komplementer, pengujian juga memverifikasi bahwa vault yang tersimpan di server sama sekali tidak mengandung *plaintext* sensitif seperti nama layanan, *username*, maupun *password*.



Gambar 4.13 Tampilan vault masih kosong

Hasil uji menunjukkan vault masih kosong karena belum ada data yang *password* yang ditambahkan.



Gambar 4.14 Tampilan vault yang diisi

Dan berikut tampilan vault ketika sudah ditambahkan data telah ditampilkan lengkap, dan seluruh nilai terverifikasi sesuai.

- **Master password salah / share invalid, dekripsi gagal**

Pengujian ini dilakukan untuk membuktikan bahwa sistem menolak akses secara konsisten dan tidak pernah mengembalikan *plaintext* vault ketika kredensial tidak valid.

Hasil Pengujian:

```
MODE NORMAL
Username: demo_user
Master password:
[VAULT] Master password salah atau local share rusak.
[!] Akses ditolak. Password salah, server bermasalah, atau vault tidak valid.
Tekan Enter untuk lanjut...
```

Gambar 4. 15 User gagal masuk karena master *password* salah

Hasil pengujian menunjukkan bahwa *user* tidak dapat masuk ke dalam vault, sehingga dekripsi vault tidak dilakukan. Serta data *password* tidak ditunjukkan.

4.4 Uji Penambahan Data Password

Tujuan pengujian ini adalah memverifikasi bahwa fitur penambahan *password* ke dalam vault berjalan dengan benar dalam dua mode input, pertama input manual oleh pengguna dan pembangkitan otomatis menggunakan CSPRNG, serta memastikan bahwa setiap penambahan menyebabkan vault dienkripsi ulang dengan nonce baru dan disinkronkan ke server maupun *backup* lokal.

Langkah Pengujian:

1. Jalankan server (server/app.py) dan client CLI (client/main.py) secara bersamaan.
2. Login mode normal dengan *username* demo_user dan *master password* yang telah terdaftar.
3. Pilih menu Tambah password, lalu pilih input manual, masukkan data layanan GitHub secara lengkap.
4. Kembali ke menu utama dan buka daftar *password* dan verifikasi entri GitHub muncul.

5. Pilih Tambah password kembali, lalu pilih generate otomatis, masukkan panjang 18 dan pilih semua kategori karakter.
6. Perlihatkan *password* yang berhasil dibangkitkan dan entri tersimpan di vault.
7. Sebelum dan sesudah penambahan, jalankan perintah berikut di terminal untuk membaca nonce vault di database server:

```
.venv\Scripts\python.exe -c "import sqlite3; conn =  
sqlite3.connect('server/pm_server.db');  
print(conn.execute(\"SELECT vault_nonce FROM users WHERE  
username='demo_user'\").fetchone()[0]); conn.close()"
```

8. Buka file lokal pengguna untuk memverifikasi backup vault ikut diperbarui:

```
type client\local_users\demo_user.json
```

Pengujian:

- **Tambah password input manual, berhasil masuk vault**

Pengguna login mode normal, memilih Tambah password input manual, dan memasukkan data sebagai berikut:

```
Nama layanan : GitHub  
Username : demo  
Password : 1234567  
Catatan : akun utama
```

Setelah konfirmasi, sistem memanggil `add_entry` yang menambahkan entri ke list vault, mengenkripsi ulang vault, dan mendorong ke server.

```
MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

Pilih aksi: 4

TAMBAH PASSWORD

Nama layanan: Github
Username/email: demo
Password untuk layanan
  1. Input manual
  2. Generate otomatis dengan CSPRNG
Pilih metode [1]: 1
Password layanan:
Catatan []: akun utama
[OK] Password ditambahkan. Vault dan backup lokal sudah diperbarui.
```

Gambar 4.16 Input *password* baru

Hasil Pengujian:

```
VAULT TERBUKA

Vault : demo_user
Mode  : NORMAL
Isi   : 1 password

MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

Pilih aksi: 3
+-----+-----+-----+-----+
| No | Layanan          | Username | Password |
+-----+-----+-----+-----+
| 1  | Github           | demo    | ***** |
+-----+-----+-----+-----+

Nomor entry: 1

+-----+-----+-----+-----+
| Detail Password |
+-----+-----+-----+-----+
Layanan : Github
Username : demo
Password : 1234567
Catatan  : akun utama
+-----+-----+-----+-----+
```

Gambar 4.17 Password baru telah masuk ke daftar password

Pengguna kemudian membuka menu Lihat daftar *password* dan entri GitHub muncul pada posisi pertama dengan seluruh *field* sesuai dengan yang dimasukkan.

- **Tambah password CSPRNG sesuai panjang, berhasil dibangkitkan**

Pengguna memilih Tambah password kemudian *generate* otomatis, memasukkan panjang 16, dan memilih semua kategori karakter aktif. Sistem memanggil `generate_password` dari `crypto/csprng.py` yang menggunakan `secrets.randbelow()` berbasis `os.urandom` untuk memilih karakter secara acak, memastikan minimal satu karakter dari setiap kategori, lalu mengacak urutan dengan Fisher-Yates shuffle.

Password yang dihasilkan ditampilkan ke pengguna sebelum disimpan. Hasil menunjukkan *password* berupa string sepanjang tepat 16 karakter yang mengandung minimal satu huruf besar, satu huruf kecil, satu angka, dan satu simbol sesuai dengan kategori yang dipilih. Entri disimpan ke vault menggunakan nilai *password* hasil CSPRNG tersebut.

```
TAMBAH PASSWORD
Nama layanan: Line
Username/email: demo
Password untuk layanan
  1. Input manual
  2. Generate otomatis dengan CSPRNG
Pilih metode [1]: 2

GENERATE PASSWORD CSPRNG
Panjang password [16]: 16
Kategori karakter
Gunakan huruf besar A-Z? [Y/n]: y
Gunakan huruf kecil a-z? [Y/n]: y
Gunakan angka 0-9? [Y/n]: y
Gunakan simbol? [Y/n]: y
[OK] Password 16 karakter berhasil dibuat.
)u?$7?.{v4zbX;K!
Catatan []:
[OK] Password ditambahkan. Vault dan backup lokal sudah diperbarui.
```

Gambar 4.18 Input password baru menggunakan CSPRNG

Hasil Pengujian:

```

          VAULT TERBUKA
Vault : demo_user
Mode  : NORMAL
Isi   : 2 password

          MENU VAULT
[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

Pilih aksi: 3
+-----+-----+-----+
| No | Layanan | Username | Password |
+-----+-----+-----+
| 1  | Github  | demo     | ***** |
| 2  | Line    | demo     | ***** |
+-----+-----+-----+

Nomor entry: 2

+-----+-----+-----+
| Detail Password |
+-----+-----+-----+
Layanan : Line
Username : demo
Password : )u?$7?.{v4zbX;K!
Catatan :
+-----+-----+-----+

```

Gambar 4.19 Input password baru menggunakan CSPRNG

Pengguna kemudian membuka menu Lihat daftar *password* dan entri Line muncul dengan *password* yang telah di *generate* sebelumnya.

- **Setelah penambahan, vault dienkripsi ulang dengan nonce baru, disimpan ke server**

Untuk membuktikan bahwa `_save_vault` selalu menggunakan nonce baru setiap enkripsi ulang, nonce vault dibaca dari database server sebelum dan sesudah pemanggilan `add_entry`.

Hasil Pengujian:

```
almafelicia@almas-macbook-air-2 tugas4-kriptografi-password-manager % .venv/bin/python -c
"
import sqlite3
conn = sqlite3.connect('server/pm_server.db')
print(conn.execute("\SELECT vault_nonce FROM users WHERE username='demo_user'").fetchone(
)[0])
conn.close()
"
b13ea6cc6159ee3a456c2ffc
```

Gambar 4.20 Hasil nonce sebelum ditambahkan *password*

```
almafelicia@almas-macbook-air-2 tugas4-kriptografi-password-manager % .venv/bin/python -c
"
import sqlite3
conn = sqlite3.connect('server/pm_server.db')
print(conn.execute("\SELECT vault_nonce FROM users WHERE username='demo_user'").fetchone(
)[0])
conn.close()
"
bf68ddf2670ce75dcab8e7a6
```

Gambar 4.21 Hasil nonce setelah ditambahkan *password*

Setelah menambahkan satu entri *password* melalui CLI, perintah yang sama dijalankan kembali. Nilai `vault_nonce` yang terbaca berbeda dari sebelumnya, membuktikan bahwa sistem selalu membangkitkan nonce 12-byte baru menggunakan `os.urandom(12)` setiap kali vault dienkripsi ulang. Ini merupakan persyaratan kritis AES-GCM di mana penggunaan nonce yang sama dengan kunci yang sama akan merusak kerahasiaan dan integritas enkripsi.

- **Backup vault lokal juga diperbarui**

Setelah `add_entry` berhasil, fungsi `_save_vault` juga memanggil `storage.save_backup_vault` yang memperbarui field `enc_backup_vault` dan `backup_vault_nonce` di file JSON lokal pengguna.

Hasil Pengujian:

```
almafeliccia@almas-macbook-air-2 tugas4-kriptografi-password-manager % cat client/local_users/demo_user.json

{
  "enc_local_share": "eao05bTyIj6YYqYR+uSZx1W0XXw6YiKE41QCYLahcCRr6/wUicuLl0ZprExqlcxrJdQ+7VdG4i0bTftwj053y5q0I1d0Pzc9FzVbso4uDgG4XupAt85EJCZLHPGuyHp3xQCGeqJFzhoV",
  "local_share_nonce": "AiYwWkGkRGP7Yfav",
  "kdf_salt": "79eDvkrri59TcGhi+H3Ljw==",
  "kdf_params": {
    "time_cost": 3,
    "memory_cost": 65536,
    "parallelism": 4,
    "hash_len": 32,
    "type": "argon2id"
  },
  "username": "demo_user",
  "enc_backup_vault": "dNzae2zTkhy610nTYbnjypRB",
  "backup_vault_nonce": "sT6mzGFZ7jpFbC/8"
}
```

Gambar 4.22 Hasil file JSON lokal sebelum ditambahkan password

Hasil menunjukkan file JSON lokal memiliki dua *field backup* yang diperbarui setelah penambahan entri.

```
almafeliccia@almas-macbook-air-2 tugas4-kriptografi-password-manager % cat client/local_users/demo_user.json

{
  "enc_local_share": "eao05bTyIj6YYqYR+uSZx1W0XXw6YiKE41QCYLahcCRr6/wUicuLl0ZprExqlcxrJdQ+7VdG4i0bTftwj053y5q0I1d0Pzc9FzVbso4uDgG4XupAt85EJCZLHPGuyHp3xQCGeqJFzhoV",
  "local_share_nonce": "AiYwWkGkRGP7Yfav",
  "kdf_salt": "79eDvkrri59TcGhi+H3Ljw==",
  "kdf_params": {
    "time_cost": 3,
    "memory_cost": 65536,
    "parallelism": 4,
    "hash_len": 32,
    "type": "argon2id"
  },
  "username": "demo_user",
  "enc_backup_vault": "WT8huoD0hhJaRl4roN6wTZWmSnwJ7goG5rZ21YsN6f/QoYiQNgio26tR07yL6I8/4VLlMuXfIm/t36/V416RRA0zLE02jkkWmpyPrUwQikkonczbAkIoaEjJk0oEs+e87IJyv8XsMN7wCBPDxqYEI+UXYVoXGJaIkM99D4o+aU9ZNzx2cxe0mWSv6u759uQgVgpcZYzSw667nFt8rxL0u3HiyY9sagjFL8CBfMT2pVsVGe/b1Kc5ZHmZEZ2PIM7THbIwLaYicNdsVkbw==",
  "backup_vault_nonce": "v2jd8mcM513Ku0em"
}
```

Gambar 4.23 Hasil file JSON lokal setelah ditambahkan password

Field `enc_backup_vault` berisi *ciphertext* vault terbaru, dan `backup_vault_nonce` menyimpan nonce enkripsi backup. Backup ini digunakan oleh `open_vault_backup` ketika server tidak dapat diakses, sehingga konsistensi antara data server dan backup lokal selalu terjaga setiap kali ada perubahan vault.

4.5 Uji Pengubahan dan Penghapusan Data Password

Tujuan pengujian ini adalah memverifikasi bahwa operasi pengubahan dan penghapusan data *password* berjalan dengan benar. Perubahan tersimpan setelah vault dienkripsi ulang dan disinkronkan ke server, entri yang dihapus tidak lagi muncul saat vault dibuka kembali, serta kedua operasi ini hanya dapat dilakukan pada mode normal dan tidak tersedia pada mode *backup*.

Langkah Pengujian:

1. Login mode normal, buka *daftar password*, dan pastikan sudah ada entri yang akan diubah/dihapus.
2. Untuk pengubahan. Pilih menu Edit password, pilih entri yang akan diubah, ganti nilai *field* yang diinginkan (misalnya *password* baru), lalu konfirmasi penyimpanan.
3. Buka daftar *password* kembali dan verifikasi nilai yang diubah sudah diperbarui.
4. Untuk penghapusan. Pilih menu Hapus password, pilih entri yang akan dihapus, dan konfirmasi penghapusan.
5. Buka daftar *password* kembali dan verifikasi entri tidak lagi muncul.
6. Untuk konfirmasi mode normal. Keluar dari aplikasi, lalu jalankan ulang dalam mode backup dan verifikasi bahwa opsi edit dan hapus tidak tersedia.

Pengujian:

- **Ubah data password, tersimpan setelah vault dienkripsi ulang**

Fungsi `edit_entry` menyimpan salinan entry lama sebagai cadangan, memperbarui hanya `field` yang diberikan argumen bukan `None`, lalu memanggil `_save_vault` untuk mengenkripsi ulang seluruh vault dengan `nonce` baru dan mendorongnya ke server. Jika `_save_vault` gagal, nilai `entry` dikembalikan ke kondisi semula sehingga vault tidak berada dalam keadaan setengah berubah.

```
UBAH PASSWORD

+-----+-----+-----+
| No | Layanan | Username | Password |
+-----+-----+-----+
| 1 | Github | demo | ***** |
| 2 | Line | demo | ***** |
+-----+-----+-----+

Nomor entry: 1
Nama layanan [Github]: Github
Username/email [demo]: demo
Password baru (kosong=pakai lama, g=generate) []: Kuubah
Catatan [akun utama]: akun sama
[OK] Password berhasil diubah dan vault dienkripsi ulang.
```

Gambar 4.24 Mengubah *password*

Hasil Pengujian:

Pengujian `test_edit_entry_updates_field` dan `test_edit_entry_all_fields` menunjukkan bahwa `edit_entry` berhasil memperbarui *field* yang ditentukan saja.

```
MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

Pilih aksi: 3

+-----+-----+-----+
| No | Layanan | Username | Password |
+-----+-----+-----+
| 1 | Github | demo | ***** |
| 2 | Line | demo | ***** |
+-----+-----+-----+

Nomor entry: 1

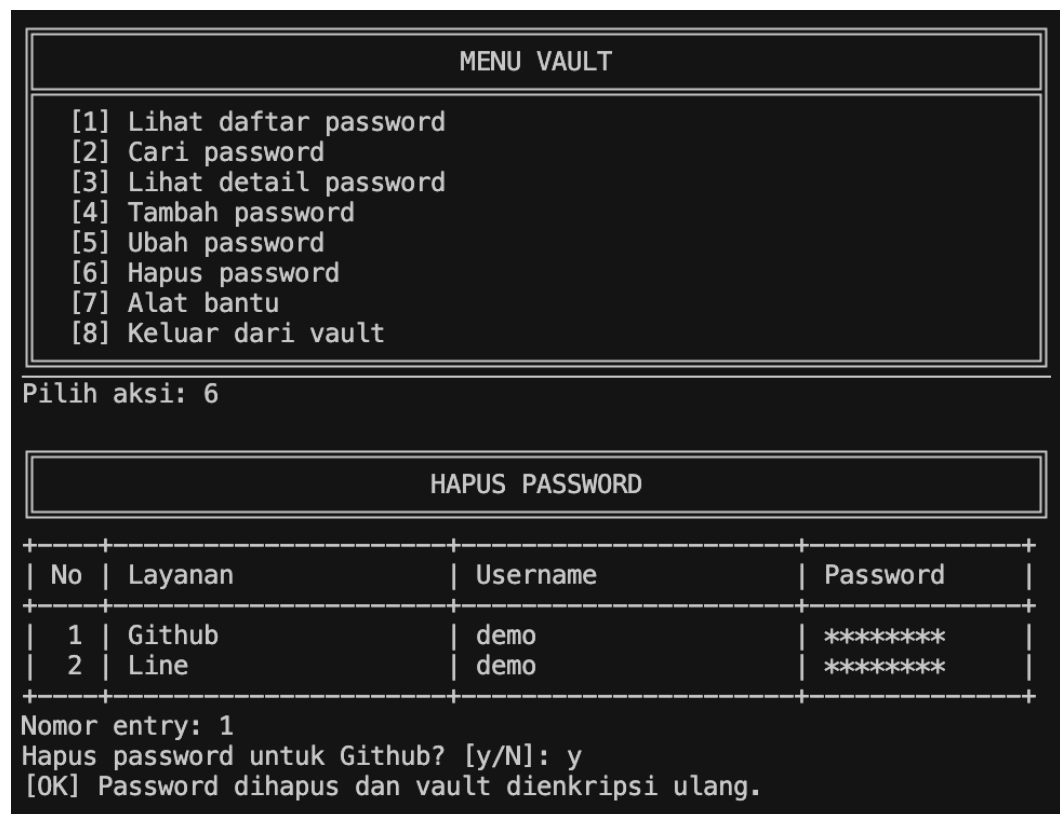
+-----+-----+-----+
| Detail Password |
+-----+-----+-----+
Layanan : Github
Username : demo
Password : Kuubah
Catatan : akun sama
+-----+-----+-----+
```

Gambar 4.25 Tampilan password yang telah diubah

Pengujian `test_edit_and_reopen_vault` membuktikan bahwa perubahan tersimpan secara permanen: ketika vault dibuka ulang menggunakan `open_vault_normal` setelah operasi edit, nilai *field* yang diubah sudah mencerminkan nilai baru. Perubahan ini konsisten karena vault yang baru dienkrpsi ulang dan didorong ke server sebelum fungsi mengembalikan hasil.

- **Hapus data password, tidak muncul setelah vault dibuka kembali**

Fungsi `delete_entry` mengeluarkan entri dari list vault menggunakan `pop(index)`, menyimpannya sebagai cadangan rollback, lalu memanggil `_save_vault` untuk mengenkripsi ulang vault yang sudah berkurang satu entri tersebut dan mendorongnya ke server. Jika *push* ke server gagal, entri yang tadi di-*pop* disisipkan kembali ke posisi semula sehingga vault tidak kehilangan data.



Gambar 4.26 Menghapus password

Hasil Pengujian:

Pengujian `test_delete_entry_removes_item` menunjukkan panjang list vault berkurang dari 2 menjadi 1 setelah `delete_entry` dipanggil.

```
VAULT TERBUKA

Vault : demo_user
Mode  : NORMAL
Isi   : 1 password

MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Tambah password
[5] Ubah password
[6] Hapus password
[7] Alat bantu
[8] Keluar dari vault

Pilih aksi: 1

+-----+-----+-----+-----+
| No | Layanan | Username | Password |
+-----+-----+-----+-----+
| 1 | Line | demo | ***** |
+-----+-----+-----+-----+
```

Gambar 4.27 Tampilan list password

Pengujian `test_delete_and_reopen_vault` mengonfirmasi bahwa entri yang dihapus tidak muncul kembali saat vault dibuka ulang menggunakan `open_vault_normal`, membuktikan bahwa vault yang baru berhasil disimpan ke server secara permanen.

- **Pembaruan hanya bisa di mode normal**

Fungsi `edit_entry` dan `delete_entry` secara internal memanggil `_save_vault`, yang bergantung pada `api.fetch_server_data` dan `api.push_vault` untuk mengambil *server share* dan mendorong vault baru. Kedua pemanggilan API ini memerlukan server aktif. Pada mode *backup* (`open_vault_backup`), server tidak diakses sama sekali tidak ada fungsi `edit_entry`, `delete_entry`, maupun `_save_vault` yang dipanggil, dan antarmuka CLI mode *backup* tidak menyediakan menu edit maupun hapus.

Hasil Pengujian:

Ketika client dijalankan dalam mode *backup*, menu utama hanya menampilkan opsi Lihat daftar password tanpa opsi edit atau hapus. Percobaan memanggil `edit_entry` atau `delete_entry` secara programatik tanpa server yang aktif sehingga `api.fetch_server_data` mengembalikan `False` yang menyebabkan `_save_vault` mengembalikan `False` dan fungsi menjalankan *rollback*, sehingga tidak ada perubahan yang tersimpan. Ini membuktikan bahwa operasi pembaruan data vault dirancang untuk hanya bisa dilakukan ketika server dapat diakses.

```
MODE BACKUP
Username: demo_user
Mode backup bersifat read-only.
Master password:
Masukkan recovery share dalam format SSS:3:<hex>.
Jika hanya punya index dan value terpisah, kosongkan input pertama.
Recovery share []: SSS:3:42d443523c36306608be5d17ed37a60d91ac624ead9583b5c5b697dc665bbaf9
[OK] Vault berhasil dibuka dengan local share + recovery share.

VAULT TERBUKA

Vault : demo_user
Mode  : BACKUP (READ-ONLY)
Isi   : 1 password
Tambah, ubah, dan hapus dinonaktifkan.

MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Alat bantu
[5] Keluar dari vault
```

Gambar 4.28 Mode backup tidak dapat mengubah/ menghapus *password*

4.6 Uji Penyimpanan Vault di Server

Tujuan pengujian ini adalah memastikan bahwa server hanya menyimpan data yang diperlukan untuk mode normal, yaitu *server share*, *vault* terenkripsi, *nonce vault*, dan metadata pengguna. Pengujian juga dilakukan untuk membuktikan bahwa server tidak menyimpan isi *vault* maupun data sensitif lain dalam bentuk *plaintext*, sesuai prinsip *zero-knowledge server*.

Langkah Pengujian:

1. Membuka database SQLite server melalui script inspeksi database. Jalankan query berikut di terminal:

```
@'
import sqlite3

USERNAME = "demo_user"

PLAINTEXT_TERMS = [
    "GitHub",
    "Gmail",
    "demo@example.com",
    "manualPass123!",
]

conn = sqlite3.connect("server/pm_server.db")
conn.row_factory = sqlite3.Row

print("=== Kolom tabel users ===")
columns = list(conn.execute("PRAGMA table_info(users)"))
for row in columns:
    print(row["name"], row["type"])

column_names = {row["name"] for row in columns}
forbidden_columns = {
    "nama_layanan", "password", "catatan",
    "master_key", "local_share", "recovery_share",
    "plaintext_vault"
}

print("\n=== Cek kolom terlarang ===")
found = forbidden_columns & column_names
print("Kolom terlarang ditemukan:", found if found else "Tidak ada")

print("\n=== Data user ===")
row = conn.execute("""
SELECT username,
       server_share,
       typeof(vault_blob) AS vault_type,
       length(vault_blob) AS vault_len,
       vault_blob,
       vault_nonce,
       created_at,
```

```

        updated_at
FROM users
WHERE username = ?
"""", (USERNAME,)).fetchone()

if row is None:
    print(f"User '{USERNAME}' tidak ditemukan.")
else:
    print("username:", row["username"])
    print("server_share:", row["server_share"])
    print("vault_type:", row["vault_type"])
    print("vault_len:", row["vault_len"])
    print("vault_nonce:", row["vault_nonce"])
    print("created_at:", row["created_at"])
    print("updated_at:", row["updated_at"])

    print("\n=== Cek plaintext data vault di server ===")
    vault_blob = row["vault_blob"]
    server_text = (
        row["username"]
        + row["server_share"]
        + row["vault_nonce"]
        + row["created_at"]
        + row["updated_at"]
    )

    for term in PLAINTEXT_TERMS:
        in_blob = term.encode("utf-8") in vault_blob
        in_text_columns = term in server_text
        print(f"{term!r}: in vault_blob={in_blob}, in text
columns={in_text_columns}")

conn.close()
'@ | .venv\Scripts\python.exe -

```

Query di atas digunakan untuk memeriksa struktur tabel `users`, memeriksa tipe data kolom `vault_blob`, mencari apakah terdapat data plaintext seperti nama layanan, `username/email`, dan `password` di dalam database server, dan memeriksa apakah terdapat kolom atau data sensitif seperti `master_key`, `local_share`, `recovery_share`, atau `plaintext_vault`.

Pengujian:

- **Vault Terenkripsi Disimpan di SQLite sebagai BLOB**

Memverifikasi bahwa vault yang disimpan pada server tidak berbentuk *plaintext*, melainkan disimpan sebagai data biner terenkripsi pada kolom `vault_blob`.

Hasil Pengujian:

```
=== Data user ===
username: demo_user
server_share: {"index": 2, "value": "0d931e508959efe6ce4078f107c955bbffdbd57eba36f8cf33911cadcc6a305c"}
vault_type: blob
vault_len: 240
vault_nonce: f22cba1ce1ccdfc0d2581e3a
created_at: 2026-06-06 09:51:59
updated_at: 2026-06-06 09:55:40
```

Gambar 4.29 Output pengecekan tipe data `vault_blob` pada database server

Berdasarkan hasil tersebut, kolom `vault_blob` pada tabel `users` memiliki tipe data `blob` dengan panjang 240 byte. Hal ini menunjukkan bahwa isi vault disimpan sebagai data biner terenkripsi, bukan sebagai JSON *plaintext* atau data *password* yang disimpan dalam baris terpisah. Dengan demikian, vault tidak dapat dibaca langsung dari database server. Data pengujian juga menunjukkan `vault_type: blob` dan `vault_len: 240`.

- **Server Tidak Menyimpan Nama Layanan, Username, Password, atau Catatan dalam Bentuk Plaintext**

Memverifikasi bahwa data akun yang sebelumnya dimasukkan ke vault tidak tersimpan sebagai *plaintext* pada database server.

Hasil Pengujian:

```
=== Cek plaintext data vault di server ===
'GitHub': in vault_blob=False, in text columns=False
'Gmail': in vault_blob=False, in text columns=False
'demo@example.com': in vault_blob=False, in text columns=False
'manualPass123!': in vault_blob=False, in text columns=False
```

Gambar 4.30 Output pengecekan plaintext data vault pada server

Berdasarkan hasil tersebut, data plaintext seperti GitHub, Gmail, demo@example.com, dan manualPass123! tidak ditemukan baik pada vault_blob maupun kolom teks server. Seluruh hasil pengecekan bernilai False. Hal ini membuktikan bahwa server tidak menyimpan nama layanan, username/email, password, maupun catatan dalam bentuk *plaintext*. Data akun pengguna hanya tersimpan sebagai bagian dari vault terenkripsi.

- **Server Tidak Menerima Master Key, Local Share, Recovery Share, maupun Plaintext Vault**

Memverifikasi bahwa server tidak memiliki kolom atau data sensitif seperti *master key*, *local share*, *recovery share*, maupun isi vault dalam bentuk *plaintext*.

Hasil Pengujian:

```
=== Kolom tabel users ===
id INTEGER
username TEXT
server_share TEXT
vault_blob BLOB
vault_nonce TEXT
created_at TEXT
updated_at TEXT

=== Cek kolom terlarang ===
Kolom terlarang ditemukan: Tidak ada
```

Gambar 4.31 Output struktur tabel users dan pengecekan kolom terlarang

Berdasarkan hasil tersebut, tabel users hanya memiliki kolom id, username, server_share, vault_blob, vault_nonce, created_at, dan updated_at. Hasil pengecekan kolom terlarang juga menunjukkan bahwa tidak ditemukan kolom seperti master_key, local_share, recovery_share, maupun plaintext_vault. Dengan demikian, server hanya menyimpan satu *share* milik server, ciphertext vault, nonce vault, dan metadata waktu. Hal ini sesuai dengan prinsip *zero-knowledge server*, karena server tidak memiliki informasi yang cukup untuk merekonstruksi master key atau membuka isi vault secara mandiri. Spesifikasi tugas juga menegaskan bahwa server tidak boleh menerima *master key*, *local share*, *recovery share*, maupun isi vault dalam bentuk *plaintext*.

4.7 Uji Mode Backup

Tujuan pengujian ini adalah memastikan bahwa mode backup dapat digunakan ketika server tidak dapat diakses. Pada mode ini, sistem harus membuka vault menggunakan kombinasi *local share* dan *recovery share*, menggunakan backup vault lokal terenkripsi sebagai sumber vault, serta hanya mengizinkan pengguna untuk melihat data tanpa melakukan penambahan, pengubahan, atau penghapusan data.

Langkah Pengujian:

1. Pastikan vault sudah memiliki data *password*.
2. Matikan server dengan `Ctrl+C` pada terminal server.
3. Jalankan aplikasi client.
4. Pilih menu `Masuk ke vault`.
5. Pilih `Mode backup`.
6. Masukkan *username demo_user*.
7. Masukkan *master password*.
8. Masukkan *recovery share*.
9. Periksa apakah data *password* dapat dilihat dan operasi tulis tidak tersedia.

Pengujian:

- **Simulasi Server Tidak Dapat Diakses**

Memverifikasi bahwa sistem dapat mendeteksi kondisi ketika server sedang tidak aktif atau tidak dapat diakses.

Hasil Pengujian:

```
(.venv) (base) PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas4-kriptografi-passwor
d-manager> .\.venv\Scripts\python.exe client\main.py

  SSS

  SHAMIR VAULT

  WELCOME TO SSS VAULT
  Distributed Password Manager CLI
  AES-128-GCM | Shamir (2,3)
  Zero-knowledge server storage

Status: belum ada vault yang sedang dibuka.

Server: offline
```

Gambar 4.32 Tampilan status server offline

Berdasarkan hasil tersebut, server berhasil disimulasikan dalam kondisi tidak dapat diakses dengan mematikan proses server melalui terminal. Pada aplikasi client, status server ditampilkan sebagai *offline*. Hal ini menunjukkan bahwa sistem berhasil mengenali kondisi server tidak tersedia, sehingga pengguna dapat menggunakan mode *backup* untuk membuka vault.

- **Vault Dibuka Menggunakan Master Password dan Recovery Share**

Memverifikasi bahwa vault pada mode *backup* dibuka menggunakan kombinasi *local share* dan *recovery share*, bukan menggunakan *server share*.

Hasil Pengujian:

```
MODE BACKUP

Username: demo_user
Mode backup bersifat read-only.
Master password:
Masukkan recovery share dalam format SSS:3:<hex>.
Jika hanya punya index dan value terpisah, kosongkan input pertama.
Recovery share []: SSS:3:145cad78ce06e7da3560b5698bae0099ea2efbfdbd809197ffa5aff69746cc23
[OK] Vault berhasil dibuka dengan local share + recovery share.
```

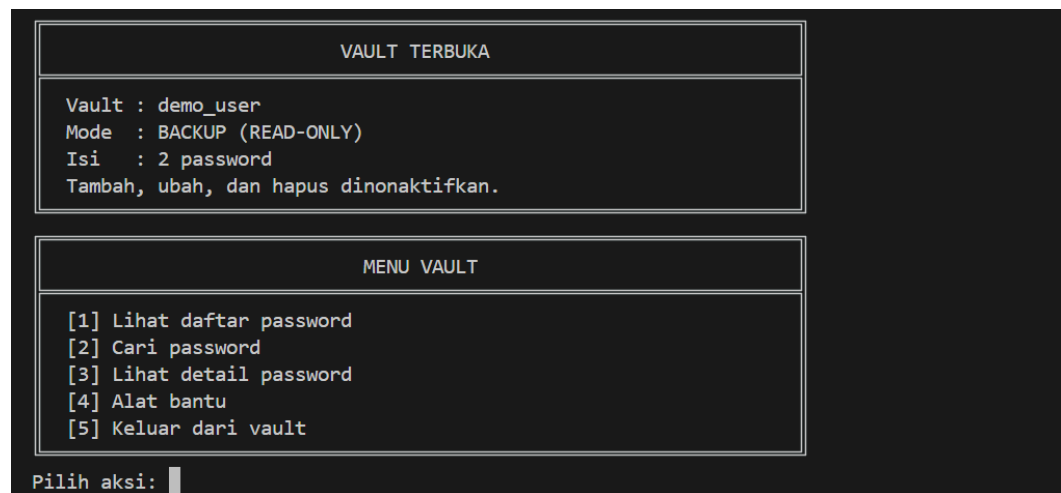
Gambar 4.33 Output vault berhasil dibuka dengan *local share* dan *recovery share*

Berdasarkan hasil pengujian, pengguna berhasil membuka vault melalui mode *backup* setelah memasukkan *username demo_user*, master password, dan *recovery share* dalam format *SSS:3:<hex>*. Sistem menampilkan pesan bahwa vault berhasil dibuka dengan *local share* dan *recovery share*. Hal ini menunjukkan bahwa pada mode *backup* sistem tidak mengambil *server share*, melainkan menggunakan dua *share* yang tersedia di sisi pengguna untuk merekonstruksi *master key*.

- **Sistem Menggunakan Backup Vault Lokal Terenkripsi**

Memverifikasi bahwa sumber vault pada mode *backup* berasal dari backup vault lokal terenkripsi yang tersimpan di sisi *client*.

Hasil Pengujian:



Gambar 4.34 Output pembukaan vault pada mode backup saat server offline

Berdasarkan hasil tersebut, vault tetap berhasil dibuka meskipun server sedang dalam keadaan *offline*. Karena server tidak dapat diakses, sistem tidak mengambil vault terenkripsi dari server. Vault yang digunakan pada mode ini berasal dari backup vault lokal terenkripsi yang tersimpan di sisi *client*. Backup tersebut kemudian didekripsi menggunakan *master key* hasil rekonstruksi dari *local share* dan *recovery share*.

- **Data Password Dapat Dilihat pada Mode Backup**

Memverifikasi bahwa data *password* yang tersimpan tetap dapat ditampilkan ketika vault dibuka melalui mode *backup*.

Hasil Pengujian:

```
MENU VAULT

[1] Lihat daftar password
[2] Cari password
[3] Lihat detail password
[4] Alat bantu
[5] Keluar dari vault

Pilih aksi: 1
+-----+-----+-----+
| No | Layanan | Username | Password |
+-----+-----+-----+
| 1 | GitHub | demo@example.com | ***** |
| 2 | Gmail | gmail@example.com | ***** |
+-----+-----+-----+

Tekan Enter untuk lanjut...
```

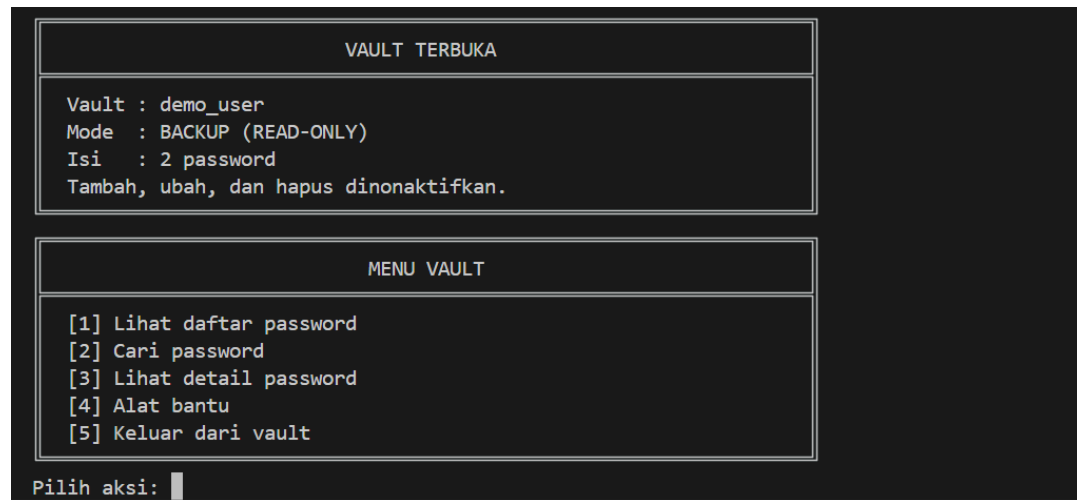
Gambar 4.35 Tampilan daftar password pada mode backup

Berdasarkan hasil pengujian, setelah vault berhasil dibuka pada mode *backup*, data *password* tetap dapat dilihat oleh pengguna. Pada pengujian ini, daftar *password* menampilkan dua data layanan, yaitu GitHub dan Gmail. Hal ini membuktikan bahwa mode *backup* dapat digunakan sebagai akses darurat untuk membaca isi vault ketika server tidak tersedia.

- **Pengguna Tidak Dapat Menambah, Mengubah, atau Menghapus Data Password pada Mode Backup**

Memverifikasi bahwa mode *backup* bersifat *read-only* dan tidak menyediakan operasi penambahan, pengubahan, atau penghapusan data *password*.

Hasil Pengujian:



Gambar 4.36 Tampilan mode *BACKUP (READ-ONLY)* dan menu tanpa operasi tulis

Berdasarkan hasil pengujian, aplikasi menampilkan keterangan Mode: *BACKUP (READ-ONLY)* serta informasi bahwa fitur tambah, ubah, dan hapus dinonaktifkan. Menu yang tersedia hanya fitur untuk melihat daftar *password*, mencari *password*, melihat detail, alat bantu, dan keluar. Tidak terdapat pilihan untuk menambah, mengubah, atau menghapus data *password*. Dengan demikian, mode *backup* berhasil dibatasi hanya untuk membaca data agar tidak terjadi konflik versi antara *backup* lokal dan vault yang tersimpan di server.

4.8 Uji Kegagalan Pemulihan

Tujuan pengujian ini adalah untuk memastikan ketahanan dan keamanan sistem pada mode backup saat mendeteksi kesalahan input dari pengguna ataupun adanya indikasi manipulasi data ilegal pada berkas enkripsi lokal. Pengujian ini bertujuan memvalidasi keandalan mekanisme matematika *Shamir Secret Sharing* (SSS) serta fitur *authenticated encryption* (AES-128-GCM) dalam menolak akses dan menyembunyikan data sensitif ketika proses pemulihan gagal.

Langkah Pengujian:

1. Mensimulasikan kondisi server tidak dapat diakses (dimatikan atau diputus koneksinya).
2. Menjalankan aplikasi klien, masuk ke menu **[1] Masuk ke vault**, lalu memilih opsi **[2] Mode backup**.
3. Memasukkan *master password* yang benar, tetapi memberikan input *recovery share* yang salah atau acak saat diminta oleh sistem
4. Membuka berkas konfigurasi lokal (`demo_user.json`) secara manual melalui *text editor*, lalu mengubah beberapa karakter pada nilai "`enc_backup_vault`" untuk mensimulasikan kerusakan atau modifikasi data.
5. Menjalankan kembali aplikasi pada Mode Backup, memasukkan seluruh kredensial secara valid, lalu memverifikasi kegagalan dekripsi akibat ketidaksesuaian tag autentikasi serta memastikan tidak ada data kata sandi yang bocor ke layar.

Pengujian:

- **Recovery share salah, rekonstruksi master key / dekripsi gagal**

Memverifikasi bahwa sistem tidak akan dapat merekonstruksi *master key* yang valid jika *recovery share* yang dimasukkan salah, sehingga proses pemulihan darurat otomatis digagalkan oleh sistem.

Hasil Pengujian:

```
client > local_users > {} demo_user.json > ...
3  "local_share_nonce": "rD6v1Ky2zb50nPNp",
4  "kdf_salt": "96VcNRzjLQgXCoFFPYr0nA==",
5  "kdf_params": {
6    "time_cost": 3,
7    "memory_cost": 65536,
8    "parallelism": 4,
9    "hash_len": 32,
10   "type": "argon2id"
11 },
12 "username": "demo_user",
13 "enc_backup_vault": "5/78uSgr1xt/OaWPiTQUVgPM",
14 "backup_vault_nonce": "BHi/f/osxFcDWzmz"
15 }

PROBLEMS 9 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

MODE BACKUP

Username: demo_user
Mode backup bersifat read-only.
Master password:
Masukkan recovery share dalam format SSS:3:<hex>.
Jika hanya punya index dan value terpisah, kosongkan input pertama.
Recovery share []: SSS:3:7a20b06da43451ddeff2dbfbfa8098bcffc15940a81d09840309ae665f951939
[VAULT] Recovery share salah atau backup vault rusak.
[!] Akses backup gagal. Password, recovery share, atau backup vault tidak valid.

Tekan Enter untuk lanjut...
```

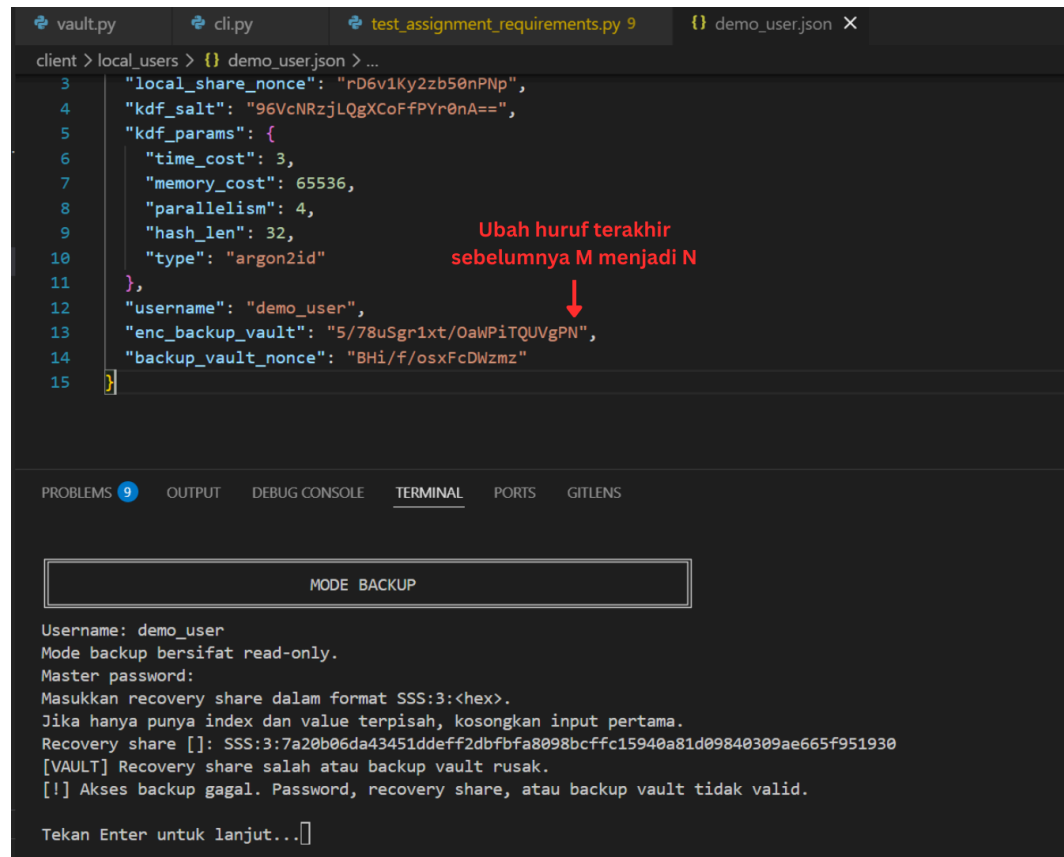
Gambar 4.37 Kegagalan Rekonstruksi Kunci Akibat Nilai Recovery Share Tidak Valid

Ketika pengguna menginput *recovery share* yang salah, sifat matematis dari skema SSS (2,3) mencegah pembentukan kembali *master key* asli. Sistem langsung mendeteksi ketidaksesuaian koordinat atau nilai komponen rahasia tersebut, menghentikan proses pemulihan, serta menampilkan pesan *error* bahwa rekonstruksi kunci gagal demi mengamankan *vault*.

- **Backup vault dimodifikasi, dekripsi AES-128-GCM gagal**

Memverifikasi bahwa algoritma AES-128-GCM pada sistem mampu mendeteksi setiap perubahan sekecil apa pun pada berkas *backup* lokal lewat pemeriksaan *authentication tag* (*auth tag*), sehingga *ciphertext* yang telah dimodifikasi akan otomatis ditolak.

Hasil Pengujian:



The screenshot shows a terminal window with a dark theme. At the top, there are tabs for 'vault.py', 'cli.py', 'test_assignment_requirements.py 9', and 'demo_user.json'. The main window displays a JSON configuration for a Vault user. A red arrow points to the 'enc_backup_vault' field, with a red text annotation above it: 'Ubah huruf terakhir sebelumnya M menjadi N' (Change the last letter from M to N). Below the JSON, the terminal output shows the Vault backup process. It starts with 'MODE BACKUP', then 'Username: demo_user', 'Mode backup bersifat read-only.', and 'Master password:'. It then prompts for a recovery share in SSS:3:hex format. The output shows a recovery share, followed by an error message: '[VAULT] Recovery share salah atau backup vault rusak.' and '[!] Akses backup gagal. Password, recovery share, atau backup vault tidak valid.' The prompt 'Tekan Enter untuk lanjut...' is at the bottom.

```
client > local_users > {} demo_user.json > ...
3  "local_share_nonce": "rD6v1Ky2zb50nPNp",
4  "kdf_salt": "96VcNRzjLQgXCoFFPYr0nA==",
5  "kdf_params": {
6    "time_cost": 3,
7    "memory_cost": 65536,
8    "parallelism": 4,
9    "hash_len": 32,
10   "type": "argon2id"
11 },
12 "username": "demo_user",
13 "enc_backup_vault": "5/78uSgr1xt/OaWPiTQUVgPN",
14 "backup_vault_nonce": "BHi/f/osxFcDWzmz"
15 }
```

Ubah huruf terakhir
sebelumnya M menjadi N

MODE BACKUP

Username: demo_user
Mode backup bersifat read-only.
Master password:
Masukkan recovery share dalam format SSS:3:<hex>.
Jika hanya punya index dan value terpisah, kosongkan input pertama.
Recovery share []: SSS:3:7a20b06da43451ddeff2dbfbfa8098bcffc15940a81d09840309ae665f951930
[VAULT] Recovery share salah atau backup vault rusak.
[!] Akses backup gagal. Password, recovery share, atau backup vault tidak valid.
Tekan Enter untuk lanjut...

Gambar 4.38 Kegagalan Dekripsi Backup Vault Akibat Deteksi Modifikasi Data

Pengujian modifikasi pada berkas *backup* lokal terbukti memicu kegagalan total pada fungsi dekripsi AES-128-GCM. Karena data telah diubah, verifikasi nilai *auth tag* yang tertanam pada skema GCM menghasilkan status tidak valid, sehingga sistem mengenali adanya anomali/integritas data yang rusak dan langsung memutus alur pembacaan berkas.

- **Data password tidak ditampilkan saat dekripsi gagal**

Sistem secara ketat menerapkan prinsip proteksi di mana tidak ada satu pun atribut data kata sandi yang ditampilkan atau bocor ke antarmuka CLI jika proses dekripsi *vault* berstatus gagal.

Hasil Pengujian:

```
MODE BACKUP

Username: demo_user
Mode backup bersifat read-only.
Master password:
Masukkan recovery share dalam format SSS:3:<hex>.
Jika hanya punya index dan value terpisah, kosongkan input pertama.
Recovery share []: SSS:3:7a20b06da43451ddeff2dbfbfa8098bcffc15940a81d09840309ae665f951930
[VAULT] Recovery share salah atau backup vault rusak.
[!] Akses backup gagal. Password, recovery share, atau backup vault tidak valid.

Tekan Enter untuk lanjut...[]
```

Gambar 4.39 Antarmuka CLI Tetap Bersih dari Informasi Sandi Setelah Terjadi Kegagalan Proses Dekripsi

Mekanisme pengkondisian pada kode program memastikan bahwa data kata sandi hanya akan dilempar ke memori antarmuka jika status dekripsi bernilai sukses penuh. Begitu salah satu tahap dekripsi gagal baik karena salah *share* maupun berkas rusak sistem langsung mengunci akses dan kembali ke menu aman tanpa menampilkan *plaintext* apa pun.

4.9 Uji Kriptografi Visual

Tujuan pengujian ini adalah memastikan bahwa fitur kriptografi visual dapat mengubah *recovery share* menjadi QR code, membagi QR tersebut menjadi dua *visual share*, lalu menggabungkannya kembali menjadi QR code yang dapat dipindai. Pengujian ini dilakukan sesuai ketentuan bonus, yaitu QR code hasil penggabungan harus memuat *recovery share* yang sama dengan input awal.

Langkah Pengujian:

1. Dari menu awal atau menu vault, pilih Alat bantu.
2. Pilih menu Buat QR dan visual shares.
3. Masukkan *recovery share* dalam format SSS:<index>:<value>.
4. Gunakan folder *output default*, yaitu
client/recovery_artifacts/demo_user.
5. Periksa file hasil pembuatan QR dan *visual share*.

6. Pilih menu Gabungkan visual shares.
7. Masukkan path `visual_share1.png` dan `visual_share2.png`.
8. Periksa file hasil rekonstruksi `reconstructed_qr.png`.
9. Pindai QR code hasil rekonstruksi dan bandingkan hasilnya dengan *recovery share* awal.

Pengujian:

- **QR Code Awal dari Recovery Share Berhasil Ditampilkan**

Memverifikasi bahwa *recovery share* dalam format teks dapat diubah menjadi QR code awal.

Hasil Pengujian:



Gambar 4.40 QR code awal dari *recovery share*

Berdasarkan hasil pengujian, sistem berhasil mengubah *recovery share* menjadi QR code awal melalui menu Alat bantu, kemudian Buat QR dan visual shares. Pada pengujian ini, *recovery share* yang digunakan adalah:

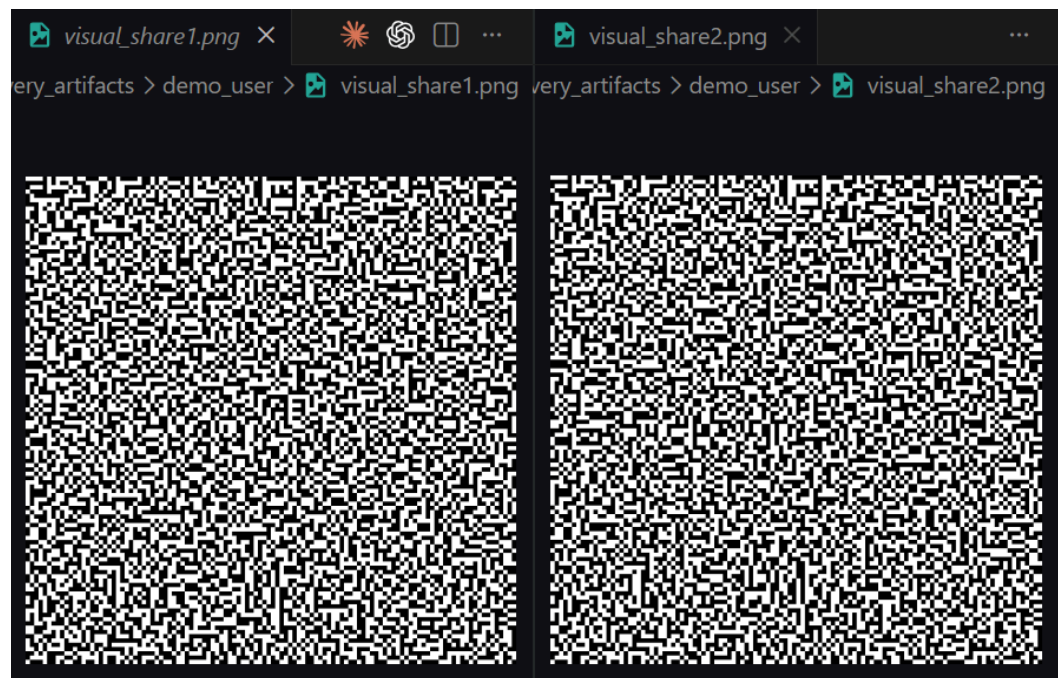
SSS:3:145cad78ce06e7da3560b5698bae0099ea2efbfdbd809197ffa5
aff69746cc23

QR code awal tersebut disimpan dalam file `client\recovery_artifacts\demo_user\recovery_qr.png`. Hal ini menunjukkan bahwa *recovery share* dapat direpresentasikan dalam bentuk QR code sebelum diproses menggunakan skema kriptografi visual.

- **Dua Gambar Visual Share Hasil Pembagian QR Code Berhasil Dibuat**

Memverifikasi bahwa QR code awal berhasil dibagi menjadi dua gambar visual share menggunakan skema *Visual Secret Sharing* (2,2).

Hasil Pengujian:



Gambar 4.41 Visual share 1 dan visual share 2 hasil pembagian QR code

Berdasarkan hasil pengujian, sistem menghasilkan dua file *visual share*, yaitu:

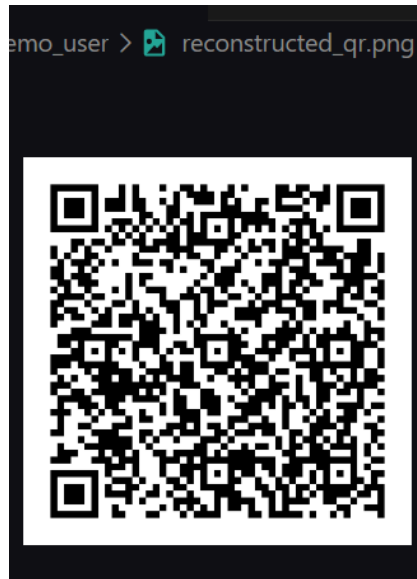
```
client\recovery_artifacts\demo_user\visual_share1.png  
client\recovery_artifacts\demo_user\visual_share2.png
```

Kedua gambar tersebut tampak seperti pola acak atau *noise*, sehingga masing-masing share tidak dapat digunakan sendiri untuk membaca *recovery share*. Hal ini sesuai dengan konsep *Visual Secret Sharing* (2,2), yaitu dua *share* harus digabungkan agar informasi rahasia dapat muncul kembali.

- **Hasil Penggabungan Kedua Visual Share Berhasil Membentuk QR Code Kembali**

Memverifikasi bahwa dua *visual share* dapat digabungkan kembali untuk menghasilkan *QR code* hasil rekonstruksi.

Hasil Pengujian:



Gambar 4.42 *QR code* hasil penggabungan *visual share*

Berdasarkan hasil pengujian, sistem berhasil menggabungkan *visual_share1.png* dan *visual_share2.png* melalui menu Gabungkan *visual shares*. Hasil penggabungan disimpan dalam file:

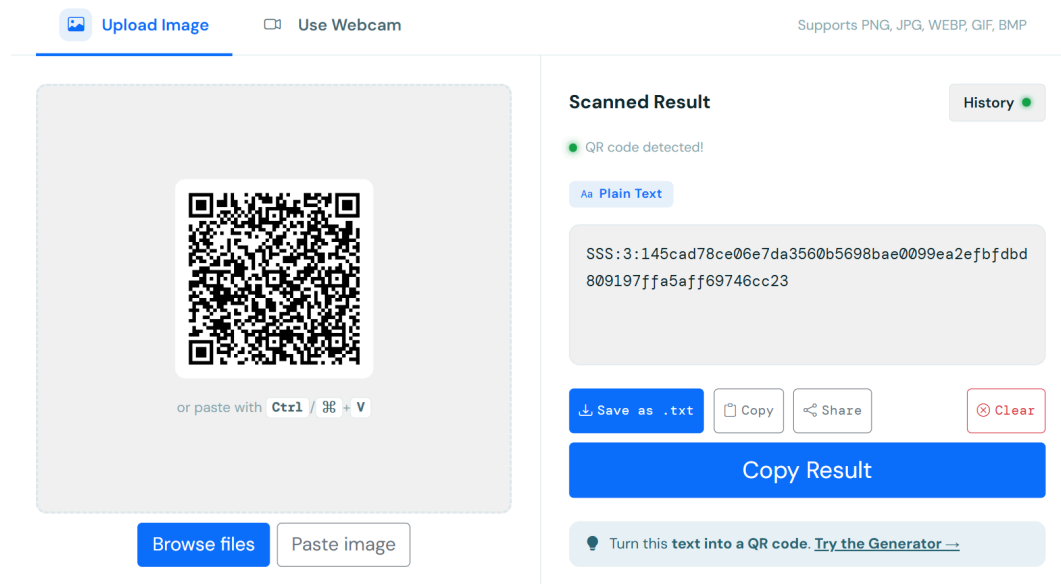
```
client\recovery_artifacts\demo_user\reconstructed_qr.png
```

QR code hasil rekonstruksi menunjukkan bahwa proses *overlay* kedua *visual share* berhasil mengembalikan bentuk *QR code* yang dapat digunakan kembali. Dengan demikian, kedua *share visual* berfungsi sebagai bagian yang saling melengkapi dalam proses pemulihan.

- **QR Code Hasil Penggabungan Dapat Dipindai dan Menghasilkan Recovery Share yang Sama**

Memverifikasi bahwa QR *code* hasil penggabungan dapat dipindai dan menghasilkan *recovery share* yang sama dengan input awal.

Hasil Pengujian:



Gambar 4.43 Hasil pemindaian QR code rekonstruksi

Berdasarkan hasil pengujian, QR *code* hasil rekonstruksi berhasil dipindai dan menghasilkan teks berikut:

```
SSS:3:145cad78ce06e7da3560b5698bae0099ea2efbfdbd809197ffa5aff69746cc23
```

Hasil tersebut sama dengan *recovery share* awal yang digunakan untuk membuat QR *code*. Dengan demikian, fitur kriptografi visual berhasil memenuhi ketentuan bonus, yaitu *recovery share* dapat diubah menjadi QR *code*, dibagi menjadi dua gambar *share*, lalu digabungkan kembali menjadi QR *code* yang dapat dipindai dan memuat *recovery share* yang sama.

V. KESIMPULAN

Aplikasi *Distributed Password Manager* berbasis *client-server* yang telah dibuat ini telah berhasil mengimplementasikan seluruh *requirement* utama dan fitur bonus secara fungsional. Melalui integrasi algoritma *Shamir Secret Sharing* (2,3), sistem ini sukses memitigasi risiko *single point of failure* (SPOF) dengan membagi *master key* menjadi *local share*, *server share*, dan *recovery share*. Dengan demikian, ketersediaan data pengguna tetap terjamin melalui Mode Backup yang bersifat *read-only* memanfaatkan *recovery share* secara mandiri saat server mengalami gangguan atau sedang *offline*. Selain itu, arsitektur ini terbukti sangat efektif dalam memenuhi prinsip *zero-knowledge server*, karena basis data server hanya menyimpan komponen terenkripsi biner beserta satu bagian *share* terisolasi tanpa pernah mengetahui *master password*, *master key*, maupun isi vault dalam bentuk teks biasa.

DAFTAR PUSTAKA

- [1] Shamir, A. (1979). How to share a secret. *Communications of the ACM*, 22(11), 612–613. <https://doi.org/10.1145/359168.359176>
- [2] National Institute of Standards and Technology (NIST). (2001). *Advanced Encryption Standard (AES)* (Federal Information Processing Standards Publication [FIPS] PUB 197). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.197>
- [3] Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication [SP] 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>
- [4] Biryukov, A., Dinu, D., & Khovratovich, D. (2016). *Argon2: New generation of memory-hard functions for password hashing and other applications*. Cryptology ePrint Archive, Report 2015/430. <https://eprint.iacr.org/2015/430>
- [5] Barker, E., & Kelsey, J. (2015). *Recommendation for Random Number Generation Using Deterministic Random Bit Generators* (NIST Special Publication [SP] 800-90A Rev. 1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-90Ar1>
- [6] Naor, M., & Shamir, A. (1995). Visual cryptography. Dalam A. De Santis (Ed.), *Advances in Cryptology EUROCRYPT '94* (pp. 1–12). Springer, Berlin, Heidelberg.
- [7] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.
- [8] Owens, M. (2006). *The Definitive Guide to SQLite*. Apress. <https://doi.org/10.1007/978-1-4302-0172-4>

LAMPIRAN

A. Pranala repository

<https://github.com/sonyaaputri/tugas4-kriptografi-password-manager>

B. Pranala video demo

 Video Demo_Kriptografi Tupil 4.MOV

C. Pembagian tugas

NIM	Nama	Tugas	
18223112	Alma Felicia Vielrizki	Kode	client/crypto/csprng.py client/main.py client/cli.py tests/test_vault.py tests/test_csprng.py requirements.txt README.md
		Laporan	BAB 3 - PERANCANGAN DAN IMPLEMENTASI 3.6 Alur Pembuatan Vault 3.7 Alur Akses Normal 3.8 Alur Akses Backup 3.10 Penambahan Data Password 3.11 Implementasi CLI BAB 4 - PENGUJIAN 4.3 Uji Akses Normal 4.4 Uji Penambahan Data Password 4.5 Uji Pengubahan dan Penghapusan Password BAB 5 - KESIMPULAN
18223131	Aulia Azka Azzahra	Kode	client/crypto/sss.py client/crypto/aes_gcm.py client/crypto/kdf.py

			client/crypto/visual_secret.py tests/test_sss.py tests/test_aes_gcm.py tests/test_kdf.py
		Laporan	BAB 2 - DASAR TEORI 2.1 Shamir Secret Sharing 2.2 AES-GCM 2.3 KDF (Argon2id) BAB 3 - PERANCANGAN DAN IMPLEMENTASI 3.2 Implementasi SSS 3.3 Implementasi AES-GCM 3.4 Implementasi KDF 3.5 Implementasi CSPRNG 3.10 Penambahan Data Password BAB 4 - PENGUJIAN 4.1 Uji Pembuatan Vault 4.2 Uji Penyimpanan Local Share 4.8 Uji Kegagalan Pemulihan
18223138	Sonya Putri Fadilah	Kode	server/app.py server/database.py server/schema.sql server/config.py client/vault.py client/api_client.py client/local_storage.py
		Laporan	BAB 1 - PENDAHULUAN BAB 2 - DASAR TEORI 2.4 CSPRNG 2.5 Kriptografi Visual BAB 3 - PERANCANGAN DAN IMPLEMENTASI 3.1 Arsitektur Sistem

			3.9 Implementasi Server 3.12 Implementasi Kriptografi Visual BAB 4 - PENGUJIAN 4.6 Uji Penyimpanan Vault di Server 4.7 Uji Mode Backup 4.9 Uji Kriptografi Visual
--	--	--	---